A Survey on Deep Semi-supervised Learning

Xiangli Yang, Zixing Song, Irwin King, *Fellow, IEEE,* Zenglin Xu, *Senior Member, IEEE*

Abstract—Deep semi-supervised learning is a fast-growing field with a range of practical applications. This paper provides a comprehensive survey on both fundamentals and recent advances in deep semi-supervised learning methods from perspectives of model design and unsupervised loss functions. We first present a taxonomy for deep semi-supervised learning that categorizes existing methods, including deep generative methods, consistency regularization methods, graph-based methods, pseudo-labeling methods, and hybrid methods. Then we offer a detailed comparison of these methods in terms of the type of losses, contributions, and architecture differences. In addition to the progress in the past few years, we further discuss some shortcomings of existing methods and provide some tentative heuristic solutions for solving these open problems.

Index Terms—Deep semi-supervised learning, image classification, machine learning, survey

1 Introduction

Deep learning has been an active research field with abundant applications in pattern recognition [1], [2], data mining [3], statistical learning [4], computer vision [5], [6], natural language processing [7], [8], *etc.* It has achieved great successes in both theory and practice [9], [10], especially in supervised learning scenarios, by leveraging a large amount of high-quality labeled data. However, labeled samples are often difficult, expensive, or time-consuming to obtain. The labeling process usually requires experts' efforts, which is one of the major limitations to train an excellent fully-supervised deep neural network. For example, in medical tasks, the measurements are made with expensive machinery, and labels are drawn from a time-consuming analysis of multiple human experts. If only a few labeled samples are available, it is challenging to build a successful learning system. By contrast, the unlabeled data is usually abundant and can be easily or inexpensively obtained. Consequently, it is desirable to leverage a large number of unlabeled data for improving the learning performance given a small number of labeled samples. For this reason, semi-supervised learning (SSL) has been a hot research topic in machine learning in the last decade [11], [12].

SSL is a learning paradigm associated with constructing models that use both labeled and unlabeled data. SSL methods can improve learning performance by using additional unlabeled instances compared to supervised learning algorithms, which can use only labeled data. It is easy to obtain SSL algorithms by extending supervised learning algorithms or unsupervised learning algorithms. SSL algorithms provide a way to explore the latent patterns from unlabeled examples, alleviating the need for a large number of labels [13]. Depending on the key objective function of the systems, one may have a semi-supervised classification, a semi-supervised clustering, or a semi-supervised regression. We provide the definitions as follows:

- **Semi-supervised classification.** Given a training dataset that consists of both labeled instances and unlabeled instances, semi-supervised classification aims to train a classifier from both the labeled and unlabeled data, such that it is better than the supervised classifier trained only on the labeled data.
- **Semi-supervised clustering.** Given a training dataset that consists of unlabeled instances, and some supervised information about the clusters, the goal of semi-supervised clustering is to obtain better clustering than the clustering from unlabeled data alone. Semi-supervised clustering is also known as constrained clustering.
- **Semi-supervised regression.** Given a training dataset that consists of both labeled instances and unlabeled instances, the goal of semi-supervised regression is to improve the performance of a regression algorithm from a regression algorithm with labeled data alone, which predicts a real-valued output instead of a class label.

In order to explain SSL clearly and concretely, we focus on the problem of image classification. The ideas described in this survey can be adapted without difficulty to other situations, such as object detection, semantic segmentation, clustering, or regression. Therefore, we primarily review image classification methods with the aid of unlabeled data in this survey.

There have been a wide variety of SSL methods, including generative models [14], [15], semi-supervised support vector machines [16], [17], graph-based methods [18], [19], [20], [21], and co-training [22]. We refer interested readers to [12], [23], which provide a comprehensive overview of traditional SSL methods. Nowadays, deep neural networks have played a dominating role in many research areas. It is important to adopt the classic SSL framework and

- *X. Yang is with the SMILE Lab, School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China*
  E-mail: xlyang@std.uestc.edu.cn
- *Z. Xu is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China, and Peng Cheng Lab, Shenzhen, China.*
  Email: xuzenglin@hit.edu.cn
- *I. King and Z. Song are with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong, China*
  Email: king@cse.cuhk.edu.hk, zxsong@cse.cuhk.edu.hk

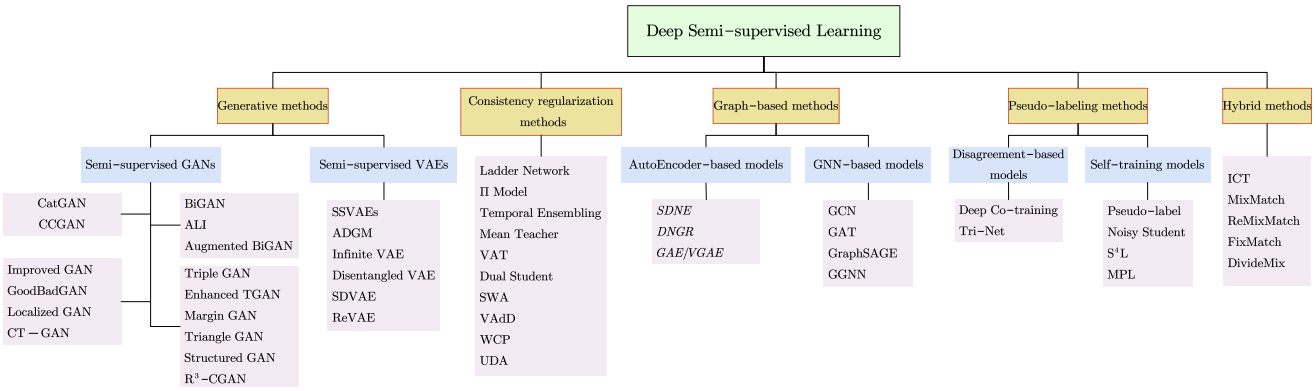

Fig. 1. The taxonomy of major deep semi-supervised learning methods based on loss function and model design.

develop novel SSL methods for deep learning settings, which leads to deep semi-supervised learning (DSSL). DSSL studies how to effectively utilize both labeled and unlabeled data by deep neural networks. A considerable amount of DSSL methods have been proposed. According to the most distinctive features in semi-supervised loss functions and model designs, we classify DSSL into five categories, *i.e.*, generative methods, consistency regularization methods, graph-based methods, pseudo-labeling methods, and hybrid methods. The overall taxonomy used in this literature is shown in Fig. 1.

There are many representative works in [12], [23], but some emerging technologies have not been included, especially after the great success of deep learning. For example, Deep semi-supervised methods proposed novel techniques like using adversarial training to generate new training data. Besides, [13] focuses on unifying the evaluation indexes of SSL, and [24] only reviews a part of SSL without making a comprehensive overview of SSL. A more recent review by Ouali *et al.* [25] gives a similar notion of DSSL as we do. However, it does not compare to the presented methods based on their taxonomy and provide perspectives on future trends and existing issues. Based on the previous research combined with the latest research, we will survey the fundamental theories and compare the deep semi-supervised methods. In summary, our contributions are listed as follows.

- We provide a detailed review of DSSL methods and introduce a taxonomy of major DSSL methods, background knowledge, and models with variants. One can quickly grasp the frontier ideas of DSSL.
- We categorize DSSL methods into generative methods, consistency regularization methods, graph-based methods, pseudo-labeling methods, and hybrid methods, with particular genres inner each one. We review the variants on each category and give standardized descriptions and unified sketch maps.
- We identify several open problems in this field and discuss the future direction for DSSL.

This survey is organized as follows. In Section 2, we introduce SSL background knowledge, including assumptions in SSL, classic SSL methods, related concepts, and datasets used in various applications. Section 4 to Section 7 introduce the primary deep semi-supervised techniques, *i.e.*, generative methods in Section 3, consistency regularization methods in Section 4, graph-based methods in Section 5, pseudo-labeling methods in Section 6 and hybrid methods in Section 7. In Section 8, we discuss the challenges in semi-supervised learning and provide some heuristic solutions and future directions for these open problems.

2 Background

In the following, we will present an overview of the techniques of SSL. Let $X = \{X_L, X_U\}$ denote the entire data set, including a small labeled data set $X_L = \{(x_i, y_i)\}_{i=1}^{L}$ with labels $Y_L = (y_1, y_2, \ldots, y_L)$ and a large scale unlabeled data set $X_U = \{(x_i)\}_{i=1}^{U}$, and $L \ll U$. We assume that the data have K classes and the first L examples within X are labeled by $\{y_i\}_{i=1}^{L} \in (y^1, y^2, \ldots, y^K)$. Formally, SSL aims to solve the following optimization problem,

$$\min_{\theta} \underbrace{\sum_{(x,y)\in X_L} \mathcal{L}_s(x, y, \theta)}_{\text{supervised loss}} + \alpha \underbrace{\sum_{(x)\in X_U} \mathcal{L}_u(x, \theta)}_{\text{unsupervised loss}} + \beta \underbrace{\sum_{(x)\in X} \mathcal{R}(x, \theta)}_{\text{regularization loss}}, \tag{1}$$

where $\mathcal{L}_s$ denote the per-example supervised loss, *e.g.*, cross-entropy for classification, $\mathcal{L}_u$ denotes the per-example unsupervised loss, and $\mathcal{R}$ denotes the per-example regularization loss, *e.g.*, consistency loss or a designed regularization term. Note that unsupervised loss terms are often not strictly distinguished from regularization terms, as normally regularization terms are also not guided by label information. Lastly, θ denotes the model parameters and $\alpha, \beta \in \mathbb{R}_{>0}$ denotes the scalar weight, which balances the loss terms. Different choices of the unsupervised loss and regularization term lead to different semi-supervised algorithms. Note that we do not make a clear distinction between unsupervised loss and regularization terms in many cases. Unless particularly specified, the notations used in this paper are illustrated in TABLE 1.

2.1 Assumptions for semi-supervised learning

SSL aims to predict more accurately with unlabeled points than supervised learning that uses only labeled data. However, an essential prerequisite is that the example distribution should be under some assumptions. If this is not the

case, SSL may not improve supervised learning and even degrade the prediction accuracy by misleading inferences. Following [23], the related assumptions in SSL are:

Semi-supervised smoothness assumption. If point x_1 is close to x_2 in a high-density region, then the corresponding outputs y_1 and y_2 are also close [23]. The hypothesis implies that if two samples belong to the same cluster, their outputs will likely be the same class label. On the contrary, they are separated by a low-density region, and then their output class labels tend to be different.

Cluster assumption. If two points x_1 and x_2 are in the same cluster, they should belong to the same category [23]. This assumption refers to the fact that each class's points tend to form a cluster, and when the points can be connected by short curves that do not pass through any low-density regions, they belong to the same class cluster [23]. According to this assumption, the decision boundary should not cross high-density areas but instead lie in low-density regions [26]. Under the premise, the learning algorithm can use a large amount of unlabeled data to adjust the classification boundary.

Low-density separation. The decision boundary should be in a low-density region, not through a high-density area [23]. We can consider the clustering assumption from another perspective by assuming that the class is separated by areas of low density [23]. Since the decision boundary in a high-density region would cut a cluster into two different classes and within such a part would violate the cluster assumption.

Manifold assumption. The input data lie on a low-dimensional manifold. If two points x_1 and x_2 are located in a local neighborhood in the low-dimensional manifold, they have similar class labels [23]. This assumption reflects the local smoothness of the decision boundary. It is well known that one of the problems of machine learning algorithms is the curse of dimensionality. It is hard to estimate the actual data distribution when volume grows exponentially with the dimensions in high dimensional spaces. If the data lie on a low-dimensional manifold, the learning algorithms can avoid the curse of dimensionality and operate in the corresponding low-dimension space. The main difference from the cluster assumption is that the cluster assumption mainly focuses on global distribution, and the manifold assumption primarily considers the locality of the data set.

TABLE 1
Notations

Symbol	Explanation
$\mathcal{X}$	Input space, for example $\mathcal{X} = \mathbb{R}^n$
$\mathcal{Y}$	Output space.
	Classification: $\mathcal{Y} = \{y^1, y^2, \ldots, y^K\}$.
	Regression: $\mathcal{Y} = \mathbb{R}$
X_L	Labeled dataset. $x_i \in \mathcal{X}, y_i \in \mathcal{Y}$
X_U	Unlabeled dataset. $x_i \in \mathcal{X}$
X	Input dataset X. $N = L + U, L \ll U$
$\mathcal{L}$	Loss Function
G	Generator
D	Discriminator
C	Classifier
H	Entropy
$\mathbb{E}$	Expectation
$\mathcal{R}$	Consistency constraint
$\mathcal{T}_x$	Consistency target
$\mathcal{G}$	A graph
$\mathcal{V}$	The set of vertices in a graph
$\mathcal{E}$	The set of edges in a graph
v	A node $v \in \mathcal{V}$
e_{ij}	An edge linked between node i and j, $e_{ij} \in \mathcal{E}$
A	The adjacency matrix of a graph
D	The degree matrix of a graph
D_{ii}	The degree of node i
W	The weight matrix
W_{ij}	The weight associated with edge e_{ij}
$\mathcal{N}(v)$	The neighbors of a node v
$\mathbf{Z}$	Embedding matrix
$\mathbf{z}_v$	An embedding for node v
$\mathbf{S}$	Similarity matrix of a graph
$\mathbf{S}[u, v]$	Similarity measurement between node u and v
$\mathbf{h}_v^{(k)}$	Hidden embedding for node v in kth layer
$\mathbf{m}_{\mathcal{N}(v)}$	Message aggregated from node v's neighborhoods

2.2 Traditional Semi-supervised learning

In the 1970s, the concept of SSL first came to the fore [27], [28], [29]. Perhaps the earliest method has been established as self-learning, an iterative mechanism in which the initial labeled data train a model to predict some unlabeled samples. The most confident prediction is marked as the best prediction of the current supervised model, thus providing more training data for the supervised algorithm. Co-training [22] provides a similar solution by training two different models on two different views. Confident predictions of one view are then used as labels for the other model. More relevant literature of this method also includes [30], [31], [32], *etc.*

An alternative technology that became famous during the "Support Vector Machine (SVM) boom" of the late 1990's was semi-supervised support vector machines [16], [17], [26], [33]. As regular SVMs, transductive support vector machines (TSVMs) optimize the gap between decision boundaries and data points. TSVMs then expand this gap based on the distance of unlabeled data to the decision margin. In [26], a smooth loss function substitutes the hinge loss of the TSVM, and for the decision boundary in a low-density space, a gradient descent technique may be used. [33] extends TSVM by utilizing an iterative method, starting with minimizing a simple convex objective function. Then more complicated procedures are used for the eventual approximation of TSVM's objective function.

Generative models assume a model $p(x, y) = p(y)p(x|y)$, where the density function $p(x|y)$ is an identifiable distribution, for example, polynomial, Gaussian mixture distribution, *etc.*, and the uncertainty is the parameters of $p(x|y)$. Generative models can be optimized by using iterative algorithms. [14], [15] apply EM algorithm for classification. They compute the parameters of $p(x|y)$ and then classify unlabeled instances according to the Bayesian full probability formula. Moreover, generative model algorithms are harsh on some assumptions. Once the hypothetical $p(x|y)$ is poorly matched with the actual distribution, it can lead to classifier performance degradation.

Graph-based methods rely on the geometry of the data induced by both labeled and unlabeled examples. This geometry is represented by an empirical graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ represent the training data points with $|\mathcal{V}| = n$ and edges $\mathcal{E}$ represent similarities between the points. By exploiting the graph or manifold structure of

data, it is possible to learn with very few labels to propagate information through the graph [18], [19], [20], [21], [34], [35]. For example, Label propagation [34] is to predict the label information of unlabeled nodes from labeled nodes. Each node label propagates to its neighbors according to the similarity.At each step of node propagation, each node updates its label according to its neighbors' label information. In the label propagation label, the label of the labeled data is fixed so that it propagates the label to the unlabeled data. The label propagation method can be applied to deep learning [36].

2.3 Related Concepts

Transfer learning. Transfer learning [37], [38], [39] aims to apply knowledge from one or more source domains to a target domain in order to improve performance on the target task. In contrast to SSL, which works well under the hypothesis that the training set and the testing set are independent and identically distributed (i.i.d.), transfer learning allows domains, tasks, and distributions used in training and testing be different but related.

Weakly supervised learning. Weakly supervised learning [40] relaxes the data dependence that requires ground-truth labels to be given for a large amount of training data set in strong supervision. There are three types of weakly supervised data: incomplete supervised data, inexact supervised data, and inaccurate supervised data. Incomplete supervised data means only a subset of training data is labeled. In this case, representative approaches are SSL and domain adaptation. Inexact supervised data suggests that the labels of training examples are coarse-grained, *e.g.*, in the scenario of multi-instance learning. Inaccurate supervised data means that the given labels are not always ground-truth, such as in the situation of label noise learning.

Meta-learning. Meta-learning [41], [42], [43], [44], [45], also known as "learning to learn", aims to learn new skills or adapt to new tasks rapidly with previous knowledge and a few training examples. It is well known that a good machine learning model often requires a large number of samples for training. The meta-learning model is expected to adapt and generalize to new environments that have been encountered during the training process. The adaptation process is essentially a mini learning session that occurs during the test but has limited exposure to new task configurations. Eventually, the adapted model can be trained on various learning tasks and optimized on the distribution of functions, including potentially unseen tasks.

Self-supervised learning. Self-supervised learning [46], [47], [48] has gained popularity due to its ability to prevent the expense of annotating large-scale datasets. It can leverage input data as supervision and use the learned feature representations for many downstream tasks. In this sense, self-supervised learning meets our expectations for efficient learning systems with fewer labels, fewer samples, or fewer trials. Since there is no manual label involved, self-supervised learning can be regarded as a branch of unsupervised learning.

3 Generative Methods

As discussed in Subsection 2.2, generative methods can learn the implicit features of the data to better model the data. They model the real data distribution from the training dataset and then generate new data with the distribution. In this section, we review the deep generative semi-supervised methods based on Generative Adversarial Network (GAN) framework and Variational AutoEncoder (VAE) framework, respectively.

3.1 Datasets and applications

SSL has many applications across different tasks and domains, such as image classification, object detection, semantic segmentation, *etc.* in the domain of computer vision, and text classification, sequence learning, *etc.* in the field of Natural Language Processing (NLP). As shown in TABLE 2, we summarize some of the most widely used datasets and representative references according to the applications area. In this survey, we mainly discuss the methods applied to image classification since these methods can be extended to other applications, such as [49] applied consistency training to object detection, [50] modified Semi-GANs to fit the scenario of semantic segmentation. There are many works that have achieved state-of-the-art performance within different applications. Most of these methods share details of the implementation and source code, and we refer the interested readers to a more detailed review of the datasets and corresponding references in TABLE 2.

3.2 Semi-supervised GANs

Since GAN can learn the distribution of real data from unlabeled samples, it can be used to facilitate SSL. There are many ways to use GAN in SSL settings. Inspired by [137], we have identified four main themes in how to use GAN for SSL, *i.e.*, (a) re-using the features from the discriminator, (b) using GAN-generated samples to regularise a classifier, (c) learning an inference model, and (d) using samples produced by a GAN as additional training data.

A typical GAN [138] consists of a generator G and a discriminator D (see Fig. 2(2)). The goal of G is to learn a distribution p_g over data x given a prior on input noise variables $p_z(z)$. The fake samples $G(z)$ generated by the generator G are used to confuse the discriminator D. The discriminator D is used to maximize the distinction between real training samples x and fake samples $G(z)$. As we can see, D and G play the following two-player minimax game with the value function $V(G, D)$:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]. \quad (2)$$

A simple SSL approach is to combine supervised and unsupervised loss during training. [139] demonstrates that a hierarchical representation of the GAN-discriminator is useful for object classification. These findings indicate that a simple and efficient SSL method can be provided by combining an unsupervised GAN value function with a supervised classification objective function, *e.g.*, $\mathbb{E}_{(x,y)\in X_l}[\log D(y|x)]$.

CatGAN. Categorical Generative Adversarial Network (CatGAN) [140] modifies the GAN's objective function to take into account the mutual information between observed examples and their predicted categorical class distributions. In particular, the optimization problem is different from

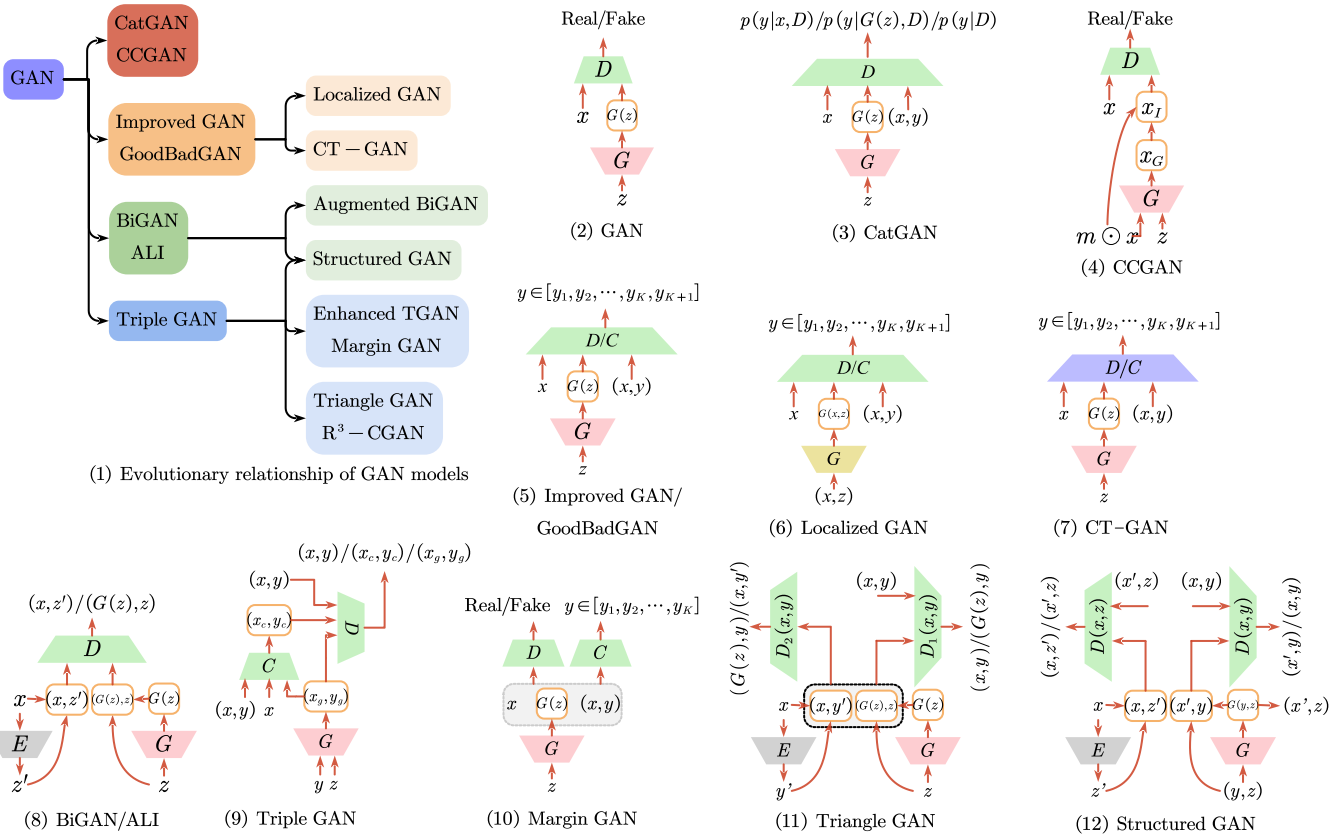

Fig. 2. A glimpse of the diverse range of architectures used for GAN-based deep generative semi-supervised methods. The characters "D, G" and "E" represent *Discriminator*, *Generator* and *Encoder*, respectively. In Figure (6), Localized GAN is equipped with a local generator $G(x, z)$, so we use the yellow box to distinguish it. Similarly, in CT-GAN, the purple box is used to denote a discriminator that introduces consistency constraint.

the standard GAN (Eq. (2)). The structure is illustrated in Fig. 2(3). This method aims to learn a discriminator which distinguishes the samples into K categories by labeling y to each x, instead of learning a binary discriminator value function. Moreover, the CatGAN discriminator loss function the supervised loss is also a cross-entropy term between the predicted conditional distribution $p(y|x, D)$ and the true label distribution of examples.n consists of three parts: (1) entropy $H[p(y|x, D)]$ which to obtain certain category assignment for samples; (2) $H[p(y|G(z), D)]$ for uncertain predictions from generated samples; (3) the marginal class entropy $H[p(y|D)]$ to uniform usage of all classes. The proposed framework uses the feature space learned by the discriminator for the final learning task. For the labeled data, the supervised loss is also a cross-entropy term between the conditional distribution $p(y|x, D)$ and the samples' true label distribution.

CCGAN. Context-Conditional Generative Adversarial Networks (CCGAN) [141] is proposed to use an adversarial loss for harnessing unlabeled image data based on image in painting. The architecture of the CCGAN is shown in Fig. 2(4). The main highlight of this work is context information provided by the surrounding parts of the image. The method trains a GAN where the generator is to generate pixels within a missing hole. The discriminator is to discriminate between the real unlabeled images and these in painted images. More formally, $m \odot x$ as input to a generator, where m denotes a binary mask to drop out a specified portion of an image and $\odot$ denotes element-wise multiplication. Thus the in painted image $x_I = (1 - m) \odot x_G + m \odot x$ with generator outputs $x_G = G(m \odot x, z)$. The in painted examples provided by the generator cause the discriminator to learn features that generalize to the related task of classifying objects. The penultimate layer of components of the discriminator is then shared with the classifier, whose cross-entropy loss is used combined with the discriminator loss.

Improved GAN. There are several methods to adapt GANs into a semi-supervised classification scenario. CatGAN [140] forces the discriminator to maximize the mutual information between examples and their predicted class distributions instead of training the discriminator to learn a binary classification. To overcome the learned representations' bottleneck of CatGAN, Semi-supervised GAN (SGAN) [142] learns a generator and a classifier simultaneously. The classifier network can have $(K + 1)$ output units corresponding to $[y_1, y_2, \ldots, y_K, y_{K+1}]$, where the y_{K+1} represents the outputs generated by G. Similar to SGAN, Improved GAN [143] solves a $(K+1)$-class classification problem. The structure of Improved GAN is shown in Fig. 2(5). Real examples for one of the first K classes and the additional $(K + 1)$th class consisted of the synthetic images generated by the generator G. This work proposes the improved techniques to train the GANs, *i.e.*, feature matching, minibatch discrimination, historical averaging one-sided label smoothing, and virtual batch normalization, where feature matching is

TABLE 2
Summary of Applications and Datasets

Applications	Datasets	Citations
Image classification	MNIST [51], SVHN [52], STL-10 [53], Cifar-10,Cifar-100 [54], ImageNet [55]	[56], [57], [58], [59], [60], [61], [62], [63], [64]
Object detection	PASCAL VOC [65], MSCOCO [66], ILSVRC [67]	[49], [68], [69], [70]
3D object detection	SUN RGB-D [71], ScanNet [72], KITTI [73]	[74], [75]
Video salient object detection	VOS [76], DAVIS [77], FBMS [78]	[79]
Semantic segmentation	PASCAL VOC [65], PASCAL context [80], MS COCO [66], Cityscapes [81], CamVid [82], SiftFlow [83], [84], StanfordBG [85]	[50], [86], [87], [88], [89], [90], [91], [92], [93], [94]
Text classification	AG News [95], BPpedia [96], Yahoo! Answers [97], IMDB [98], Yelp review [95], Snippets [99], Ohsumed [100], TagMyNews [101], MR [102], Elec [103], Rotten Tomatoes [102], RCV1 [104]	[105], [106], [107], [108], [109], [110]
Dialogue policy learning	MultiWOZ [111]	[112]
Dialogue generation	Cornell Movie Dialogs Corpus [113], Ubuntu Dialogue Corpus [114]	[115]
Sequence learning	IMDB [98], Rotten Tomatoes [102], 20 Newsgroups [116], DBpedia [96], CoNLL 2003 NER task [117], CoNLL 2000 Chunking task [118], Twitter POS Dataset [119], [120], Universal Dependencies(UD) [121], Combinatory Categorial Grammar (CCG) supertagging [122]	[123], [124], [125], [126]
Semantic role labeling	CoNLL-2009 [127], CoNLL-2013 [128]	[129], [130], [131]
Question answering	SQuAD [132], TriviaQA [133]	[134], [135], [136]

used to train the generator. It is trained by minimizing the discrepancy between features of the real and the generated examples, that is $\|\mathbb{E}_{x\in X} D(x) - \mathbb{E}_{z\sim p(z)} D(G(z))\|_2^2$, rather than maximizing the likelihood of its generated examples classified to K real classes. The loss function for training the classifier becomes

$$\max_D \mathbb{E}_{(x,y)\sim p(x,y)} \log p_D(y|x, y \le K) + \mathbb{E}_{x\sim p(x)} \log p_D(y \le K|x) + \mathbb{E}_{x\sim p_G} \log p_D(y = K+1|x), \quad (3)$$

where the first term of Eq. (3) denotes the supervised cross entropy loss, The last two terms of Eq. (3) are the unsupervised losses from the unlabeled and generated data, respectively.

GoodBadGAN. GoodBadGAN [144] realizes that the generator and discriminator in [143] may not be optimal simultaneously, *i.e.*, the discriminator achieves good performance in SSL, while the generator may generate visually unrealistic samples. The structure of GoodBadGAN is shown in Fig. 2(5). The method gives theoretical justifications of why using bad samples from the generator can boost the SSL performance. Generally, the generated samples, along with the loss function (Eq. (3)), can force the boundary of the discriminator to lie between the data manifolds of different categories, which improves the generalization of the discriminator. Due to the analysis, GoodBadGAN learns a bad generator by explicitly adding a penalty term $\mathbb{E}_{x\sim p_G} \log p(x)\mathbb{I}[p(x) > \epsilon]$ to generate bad samples, where $\mathbb{I}[\cdot]$ is an indicator function and ϵ is a threshold, which ensures that only high-density samples are penalized while low density samples are unaffected. Further, to guarantee the strong true-fake belief in the optimal conditions, a conditional entropy term $\mathbb{E}_{x\sim p_x} \sum_{k=1}^{K} \log p_D(k|x)$ is added to the discriminator objective function in Eq. (3).

Localized GAN. Localized GAN [145] focuses on using local coordinate charts to parameterize local geometry of data transformations across different locations manifold rather than the global one. This work suggests that Localized GAN can help train a locally consistent classifier by exploring the manifold geometry. The architecture of Localized GAN is shown in Fig. 2(6). Like the methods introduced in [143], [144], Localized GAN attempts to solve the $K+1$ classification problem that outputs the probability of x being assigned to a class. For a classifier, $\nabla_z D(G(x,z))$ depicts the change of the classification decision on the manifold formed by $G(x,z)$, and the evolution of $D(\cdot)$ restricted on $G(x,z)$ can be written as

$$|D(G(x, z+\sigma z)) - D(G(x,z))|^2 \approx \|\nabla_x^G D(x)\|^2 \sigma z, \quad (4)$$

which shows that penalizing $\|\nabla_x^G D(x)\|^2$ can train a robust classifier with a small perturbation σz on a manifold. This probabilistic classifier can be introduced by adding $\sum_{k=1}^{K} \mathbb{E}_{x\sim p_x} \|\nabla_x^G \log p(y = k|x)\|^2$, where $\nabla_x^G \log p(y = k|x)$ is the gradient of the log-likelihood along with the manifold $G(x,z)$.

CT-GAN. CT-GAN [146] combines consistency training with WGAN [147] applied to semi-supervised classification problems. And the structure of CT-GAN is shown in Fig. 2(7). Following [148], this method also lays the Lipschitz continuity condition over the manifold of the real data to improve the improved training of WGAN. Moreover, CT-GAN devises a regularization over a pair of samples drawn near the manifold following the most basic definition of the 1-Lipschitz continuity. In particular, each real instance x is perturbed twice and uses a Lipschitz constant to bound the difference between the discriminator's responses to the perturbed instances x', x''. Formally, since the value function of WGAN is

$$\min_G \max_D \mathbb{E}_{x\sim p_x} D(x) - \mathbb{E}_{z\sim p_z} D(G(z)), \quad (5)$$

where D is one of the sets of 1-Lipschitz function. The objective function for updating the discriminator include: (a) basic Wasserstein distance in Eq. (5), (b) gradient penalty $GP|_{\hat{x}}$ [148] used in the improved training of WGAN, where $\hat{x} = \epsilon x + (1-\epsilon)G(z)$, and (c) A consistency regularization $CT|_{x',x''}$. For semi-supervised classification, CT-GAN uses the Eq. (3) for training the discriminator instead of

the Eq.(5), and then adds the consistency regularization $CT|_{x',x''}$.

BiGAN. Bidirectional Generative Adversarial Networks (BiGANs) [149] is an unsupervised feature learning framework. The architecture of BiGAN is shown in Fig. 2(8). In contrast to the standard GAN framework, BiGAN adds an encoder E to this framework, which maps data x to z', resulting in a data pair (x, z'). The data pair (x, z') and the data pair generated by generator G constitute two kinds of true and fake data pairs. The BiGAN discriminator D is to distinguish the true and fake data pairs. In this work, the value function for training the discriminator becomes,

$$\min_{G,E}\max_{D} V(D,E,G) = \mathbb{E}_{x\sim\mathcal{X}}\underbrace{\left[\mathbb{E}_{z\sim p_E(\cdot|x)}\left[\log D\left(x,z\right)\right]\right]}_{\log D(x,E(x))} + \mathbb{E}_{z\sim p(z)}\underbrace{\left[\mathbb{E}_{x\sim p_G(\cdot|z)}\left[\log\left(1-D\left(x,z\right)\right)\right]\right]}_{\log(1-D(G(z),z))}. \quad (6)$$

ALI. Adversarially Learned Inference (ALI) [150] is a GAN-like adversarial framework based on the combination of an inference network and a generative model. This framework consists of three networks, a generator, an inference network and a discriminator. The generative network G is used as a decoder to map latent variables z (with a prior distribution) to data distribution $x' = G(z)$, which can be formed as joint pairs (x', z). The inference network E attempts to encode training samples x to latent variables $z' = E(x)$, and joint pairs (x, z') are similarly drawn. The discriminator network D is required to distinguish the joint pairs (x, z') from the joint pairs (x', z). As discussed above, the central architecture of ALI is regarded as similar to the BiGAN's (see Fig. 2(8). In semi-supervised settings, this framework adapts the discriminator network proposed in [143], and shows promising performance on semi supervised benchmarks on SVHN and CIFAR 10. The objective function can be rewritten as an extended version similar to Eq. (6).

Augmented BiGAN. Kumar *et al.* [151] propose an extension of BiGAN called Augmented BiGAN for SSL. This framework has a BiGAN-like architecture that consists of an encoder, a generator and a discriminator. Since trained GANs produce realistic images, the generator can be considered to obtain the tangent spaces of the image manifold. The estimated tangents infer the desirable invariances which can be injected into the discriminator to improve SSL performance. In particular, the Augmented BiGAN uses feature matching loss [143], $\|\mathbb{E}_{x\in X}D(E(x),x) - \mathbb{E}_{z\sim p(z)}D(z,G(z))\|_2^2$ to optimize the generator network and the encoder network. Moreover, to avoid the issue of class-switching (the class of $G(E(x))$ changed during the decoupled training), a third pair $(E(x), G(E(x)))$ loss term $\mathbb{E}_{x\sim p(x)}[\log(1 - D(E(x), G_x(E(x))))]$ is added to the objective function Eq. (6).

Triple GAN. Triple GAN [152] is presented to address the issue that the generator and discriminator of GAN have incompatible loss functions, *i.e.*, the generator and the discriminator can not be optimal at the same time [143]. The problem has been mentioned in [144], but the solution is different. As shown in Fig. 2(9), the Triple GAN tackles this problem by playing a three player game. This three-player framework consists of three parts, a generator G using a conditional network to generate the corresponding fake samples for the true labels, a classifier C that generates pseudo labels for given real data, and a discriminator D distinguishing whether a data-label pair is from the real-label dataset or not. This Triple GAN loss function may be written as

$$\begin{aligned}\min_{C,G}\max_{D} V(C,G,D) = {} & \mathbb{E}_{(x,y)\sim p(x,y)}\left[\log D\left(x,y\right)\right] \\ & + \alpha\mathbb{E}_{(x,y)\sim p_c(x,y)}\left[\log\left(1-D\left(x,y\right)\right)\right] \\ & + (1-\alpha)\,\mathbb{E}_{(x,y)\sim p_g(x,y)}\left[\log\left(1-D\left(G\left(y,z\right),y\right)\right)\right],\end{aligned} \quad (7)$$

where D obtains label information about unlabeled data from the classifier C and forces the generator G to generate the realistic image-label samples.

Enhanced TGAN. Based on the architecture of Triple GAN [153], Enhanced TGAN [154] modifies the Triple-GAN by re-designing the generator loss function and the classifier network. The generator generates images conditioned on class distribution and is regularized by class-wise mean feature matching. The classifier network includes two classifiers that collaboratively learn to provide more categorical information for generator training. Additionally, a semantic matching term is added to enhance the semantics consistency with respect to the generator and the classifier network. The discriminator D learned to distinguish the labeled data pair (x, y) from the synthesized data pair $(G(z), \tilde{y})$ and predicted data pair $(x, \bar{y})$. The corresponding objective function is similar to Eq. (7), where $(G(z), \tilde{y})$ is sampling from the pre-specified distribution p_g, and $(x, \bar{y})$ denotes the predicted data pair determined by $p_c(x)$.

MarginGAN. MarginGAN [155] is another extension framework based on Triple GAN [153]. From the perspective of classification margin, this framework works better than Triple GAN when used for semi-supervised classification. The architecture of MarginGAN is presented in Fig. 2(10). MarginGAN includes three components like Triple GAN, a generator G which tries to maximize the margin of generated samples, a classifier C used for decreasing margin of fake images, and a discriminator D trained as usual to distinguish real samples from fake images. This method solves the problem that performance is damaged due to the inaccurate pseudo label in SSL, and improves the accuracy rate of SSL.

Triangle GAN. Triangle Generative Adversarial Network ($\triangle$-GAN) [153] introduces a new architecture to match cross-domain joint distributions. The architecture of the $\triangle$-GAN is shown in Fig. 2(11). The $\triangle$-GAN can be considered as an extended version of BiGAN [149] or ALI [150]. This framework is a four-branch model that consists of two generators E and G, and two discriminators D_1 and D_2. The two generators can learn two different joint distributions by two-way matching between two different domains. At the same time, the discriminators are used as an implicit ternary function, where D_2 determines whether the data pair is from (x, y') or from $(G(z), y)$, and D_1 distinguishes real data pair (x, y) from the fake data pair $(G(z), y)$.

Structured GAN. Structured GAN [156] studies the problem of semi-supervised conditional generative modeling based on designated semantics or structures. The architecture of Structured GAN (see Fig. 2(12)) is similar to Triangle GAN [153]. Specifically, Structured GAN assumes

that the samples x are generated conditioned on two independent latent variables, *i.e.*, y that encodes the designated semantics and z contains other variation factors. Training Structured GAN involves solving two adversarial games that have their equilibrium concentrating at the true joint data distributions $p(x,z)$ and $p(x,y)$. The synthesized data pair (x',y) and (x',z) are generated by generator $G(y,z)$, where (x',y) mix real sample pair (x,y) together as input for training discriminator $D(x,y)$ and $(x'z)$ blends the E's outputs pair (x,z') for discriminator $D(x,z)$.

***R*^{3}-CGAN.** Liu *et al.* [157] propose a Class-conditional GAN with Random Regional Replacement (R3-regularization) technique, called R^3-CGAN. Their framework and training strategy relies on Triangle GAN [153]. The R^3-CGAN architecture comprises four parts, a generator G to synthesize fake images with specified class labels, a classifier C to generate instance-label pairs of real unlabeled images with pseudo-labels, a discriminator D_1 to identify real or fake pairs, and another discriminator D_2 to distinguish two types of fake data. Specifically, CutMix [158], a Random Regional Replacement strategy, is used to construct two types of between-class instances (cross-category instances and real-fake instances). These instances are used to regularize the classifier C and discriminator D_1. Through the minimax game among the four players, the class-specific information is effectively used for the downstream.

Summary. Comparing with the above discussed Semi-GANs methods, we find that the main difference lies in the number and type of the basic modules, such as generator, encoder, discriminator, and classifier. As shown in Fig. 2(1), we find the evolutionary relationship of the Semi-GANs models. Overall, CatGAN [140] and CCGAN [141] extend the basic GAN by including additional information in the model, such as category information and in-painted images. Based on Improved GAN [143], Localized GAN [145] and CT-GAN [146] consider the local information and consistency regularization, respectively. BiGAN [149] and ALI [150] learn an inference model during the training process by adding an Encoder module. In order to solve the problem that the generator and discriminator can not be optimal at the same time, Triple-GAN [152] adds an independent classifier instead of using a discriminator as a classifier.

3.3 Semi-supervised VAE

There are three reasons for latent-variable models useful for SSL: (a) It is a natural way to incorporate unlabeled data, (b) The ability to disentangle representations via the configuration of latent variables, and (c) It also allow us to use variational neural methods. Variational AutoEncoders (VAEs) [159], [160] are flexible models which combine deep autoencoders with generative latent-variable models. The generative model captures representations of the distributions rather than the observations of the dataset, and defines the joint distribution in the form of $p(x,z) = p(z)p(x|z)$, where $p(z)$ is a prior distribution over latent variables z. Since the true posterior $p(z|x)$ is generally intractable, the generative model is trained with the aid of an approximate posterior distribution $q(z|x)$. The architecture of VAEs is a two-stage network, an encoder to construct a variational approximation $q(z|x)$ to the posterior $p(z|x)$, and a decoder to parameterize the likelihood $p(x|z)$. The variational approximation of the posterior maximizes the marginal likelihood, and the evidence lower bound (ELBO) may be written as

$$\log p(x) = \log \mathbb{E}_{q(z|x)}[\frac{p(z)p(x|z)}{q(z|x)}] \tag{8}$$

$$\geq \mathbb{E}_{q(z|x)}[\log p(z)p(x|z) - \log q(z|x)]. \tag{9}$$

SSVAEs. Semi-supervised VAEs (SSVAEs) [161] develop SSL algorithms based on deep generative models, using two VAE-based generative models to latent encoder representation. The latent-feature discriminative model, referred to M1 [161], can provide more robust latent features with a deep generative model of the data. As shown in Fig. 3(1), $p_\theta(x|z)$ is a non-linear transformation, *e.g.*, a deep neural network. Latent variables z can be selected as a Gaussian distribution or a Bernoulli distribution. An approximate sample of the posterior distribution on the latent variable $q_\phi(z|x)$ is used as the classifier feature for the class label y. Generative semi-supervised model, referred to M2 [161], describes the data generated by a latent class variable y and a continuous latent variable z, is expressed as $p_\theta(x|z,y)p(z)p(y)$ (as depicted in Fig. 3(2)). $p(y)$ is the multinomial distribution, where the class labels y are treated as latent variables for unlabeled data. $p_\theta(x|z,y)$ is a suitable likelihood function. The inferred posterior distribution $q_\phi(z|y,x)$ can predict any missing labels.

Stacked generative semi-supervised model, called M1+M2, uses the generative model M1 to learn the new latent representation z_1, and uses the embedding from z_1 instead of the raw data x to learn a generative semi-supervised model M2. As shown in Fig. 3(3), the whole process can be abstracted as follow:

$$p_\theta(x,y,z_1,z_2) = p(y)p(z_2)p_\theta(z_1|y,z_2)p_\theta(x|z_1), \tag{10}$$

where $p_\theta(z_1|y,z_2)$ and $p_\theta(x|z_1)$ are parameterised as deep neural networks. In all the above models, $q_\phi(z|x)$ is used to approximate the true posterior distribution $p(z|x)$, and following the variational principle, the boundary approximation lower bound of the model is derived to ensure that the approximate posterior probability is as close to the true posterior probability as possible.

ADGM. Auxiliary Deep Generative Models (ADGM) [162] extends SSVAEs [161] with auxiliary variables, as depicted in Fig. 3(4). The auxiliary variables can improve the variational approximation and make the variational distribution more expressive by training deep generative models with multiple stochastic layers.

Adding the auxiliary variable a leaves the generative model of x,y unchanged while significantly improving the representative power of the posterior approximation. An additional inference network is introduced such that:

$$q_\phi(a,y,z|x) = q_\phi(z|a,y,x)q_\phi(y|a,x)q_\phi(a|x). \tag{11}$$

The framework has the generative model p defined as $p_\theta(a)p_\theta(y)p_\theta(z)p_\theta(x|z,y)$, where a,y,z are the auxiliary variable, class label, and latent features, respectively. Learning the posterior distribution is intractable. Thus we define the approximation as $q_\phi(a|x)q_\phi(z|y,x)$ and a classifier $q_\phi(y|a,x)$. The auxiliary unit a actually introduces a class-specific latent distribution between x and y,

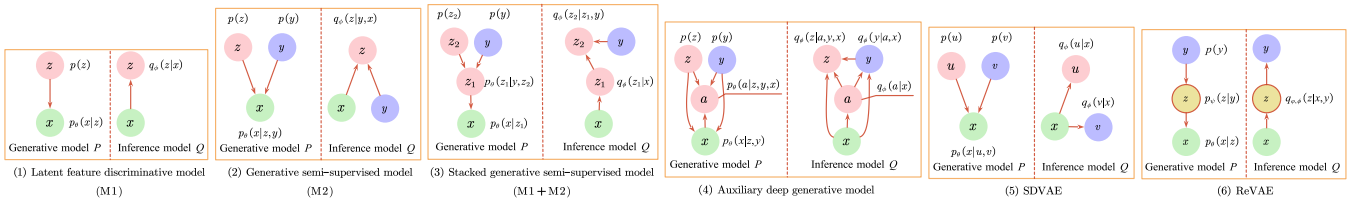

Fig. 3. A glimpse of probabilistic graphical models used for VAE-based deep generative semi-supervised methods. Each method contains two models, the generative model P and the inference model Q. The variational parameters θ and ϕ are learned jointly by the incoming connections (*i.e.*, deep neural networks).

resulting in a more expressive distribution $q_\phi(y|a,x)$. Formally, [162] employs the similar variational lower bound $\mathbb{E}_{q_\phi(a,z|x)}[\log p_\theta(a,x,y,z) - \log q_\phi(a,z|x,y)]$ on the marginal likelihood, with $q_\phi(a,z|x,y) = q_\phi(a|x)q_\phi(z|y,x)$. Similarly, the unlabeled ELBO is $\mathbb{E}_{q_\phi(a,y,z|x)}[\log p_\theta(a,x,y,z) - \log q_\phi(a,y,z|x)]$ with $q_\phi(a,y,z|x) = q_\phi(z|y,x)q_\phi(y|a,x)q_\phi(a|x)$.

Interestingly, by reversing the direction of the dependence between x and a, a model similar to the stacked version of M1 and M2 is recovered (Fig. 3(3)), with what the authors denote skip connections from the second stochastic layer and the labels to the inputs x. In this case the generative model is affected, and the authors call this the Skip Deep Generative Model (SDGM). This model is able to be trained end to end using stochastic gradient descent (SGD) (according to the [162] the skip connection between z and x is crucial for training to converge). Unsurprisingly, joint training for the model improves significantly upon the performance presented in [161].

Infinite VAE. Infinite VAE [163] proposes a mixture model for combining variational autoencoders, a non-parametric Bayesian approach. This model can adapt to suit the input data by mixing coefficients by a Dirichlet process.It combines Gibbs sampling and variational inference that enables the model to learn the input's underlying structures efficiently. Formally, Infinite VAE employs the mixing coefficients to assist SSL by combining the unsupervised generative model and a supervised discriminative model. The infinite mixture generative model as,

$$p(c,\pi,x,z) = p(c|\pi)p_\alpha(\pi)p_\theta(x|c,z)p(z), \tag{12}$$

where c denotes the assignment matrix for each instance to a VAE component where the VAE i can best reconstruct instance i. π is the mixing coefficient prior for c, drawn from a Dirichlet distribution with parameter α. Each latent variable z_i in each VAE is drawn from a Gaussian distribution.

Disentangled VAE. Disentangled VAE [164] attempts to learn disentangled representations using partially specified graphical model structures and distinct encoding aspects of the data into separate variables. It explores the graphical model for modeling a general dependency on observed and unobserved latent variables with neural networks, and a stochastic computation graph [165] is used to infer with and train the resultant generative model. For this purpose, importance sampling estimates are used to maximize the lower bound of both the supervised and semi-supervised likelihoods. Formally, this framework considers the conditional probability $q_{y,z|x}$, which has a factorization $q_\phi(y,z|x) = q_\phi(y|x,z)q_\phi(z|x)$ rather than $q_\phi(y,z|x) = q_\phi(z|x,y)q_\phi(y|x)$ in [161], which means we can no longer compute a simple Monte Carlo estimator by sampling from the unconditional distribution $q_\phi(z|x)$. Thus, the variational lower bound for supervised term expand below,

$$\mathbb{E}_{q_\phi(z|x,y)}[\log p_\theta(x|y,z)p(y)p(z) - q_\phi(y,z|x)]. \tag{13}$$

SDVAE. Semi-supervised Disentangled VAE (SDVAE) [166] incorporates the label information to the latent representations by encoding the input disentangled representation and non-interpretable representation. The disentangled variable captures categorical information, and the non-interpretable variable consists of other uncertain information from the data. As shown in Fig. 3(5), SDVAE assumes the disentangled variable v and the non-interpretable variable u are independent condition on x, *i.e.*, $q_\phi(u,v|x) = q_\phi(u|x)q_\phi(v|x)$. This means that $q_\phi(v|x)$ is the encoder for the disentangled representation, and $q_\phi(u|x)$ denotes the encoder for non-interpretable representation. Based on those assumptions, the variational lower bound is written as:

$$\mathbb{E}_{q(u|x),q(v|x)}[\log p(x|u,v)p(v)p(u) - \log q(u|x)q(v|x)]. \tag{14}$$

ReVAE. Reparameterized VAE (ReVAE) [167] develops a novel way to encoder supervised information, which can capture label information through auxiliary variables instead of latent variables in the prior work [161]. The graphical model is illustrated in Fig. 3(6). In contrast to SS-VAEs, ReVAE captures meaningful representations of data with a principled variational objective. Moreover, ReVAE carefully designed the mappings between auxiliary and latent variables. In this model, a conditional generative model $p_\psi(z|y)$ is introduced to address the requirement for inference at test time. Similar to [161] and [162], ReVAE treats y as a known observation when the label is available in the supervised setting, and as an additional variable in the unsupervised case. In particular, the latent space can be partitioned into two disjoint subsets under the assumption that label information captures only specific aspects.

Summary. As the name indicates, Semi-supervised VAE applies the VAE architecture for handling SSL problems. An advantage of these methods is that meaningful representations of data can be learned by the generative latent-variable models. The basic framework of these Semi-supervised VAE methods is M2 [161]. On the basis of the M2 framework, ADGM [162] and ReVAE [167] consider introducing additional auxiliary variables, although the roles of the auxiliary variables in the two models are different. Infinite VAE [163] is a hybrid of several VAE models to improve the performance of the entire framework. Disentangled VAE [164] and SDVAE [166] solve the semi-supervised VAE problem

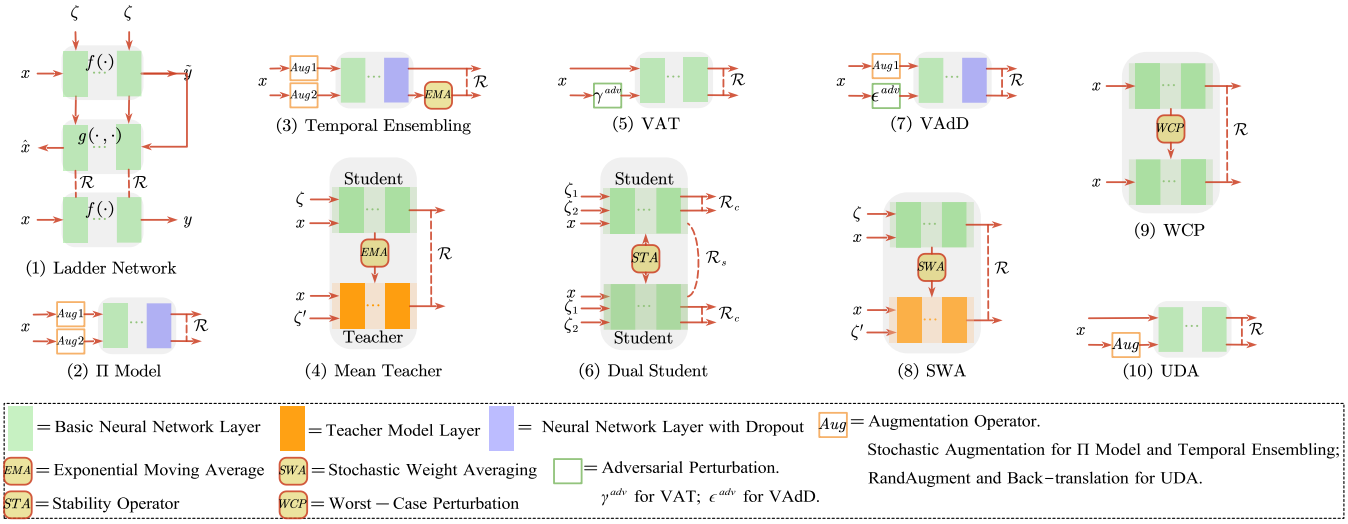

Fig. 4. A glimpse of the diverse range of architectures used for consistency regularization semi-supervised methods. In addition to the identifiers in the figure, ζ denotes the perturbation noise, and $\mathcal{R}$ is the consistency constraint.

~~by different disentangled methods. Under semi-supervised conditions, when a large number of labels are unobserved, the key to this kind of method is how to deal with the latent variables and label information.~~

4 Consistency Regularization

In this section, we introduce the consistency regularization methods for semi-supervised deep learning. In these methods, a consistency regularization term is applied to the final loss function to specify the prior constraints assumed by researchers. Consistency regularization is based on the manifold assumption or the smoothness assumption, and describes a category of methods that the realistic perturbations of the data points should not change the output of the model [13]. Consequently, consistency regularization can be regarded to find a smooth manifold on which the dataset lies by leveraging the unlabeled data [168].

The most common structure of consistency regularization SSL methods is the Teacher-Student structure. As a student, the model learns as before, and as a teacher, the model generates targets simultaneously. Since the model itself generates targets, they may be incorrect and then used by themselves as students for learning. In essence, the consistency regularization methods suffer from confirmation bias [59], a risk that can be mitigated by improving the target's quality. Formally, following [169], we assume that dataset X consists of a labeled subset X_l and an unlabeled subset X_u. Let θ' denote the weight of the target, and θ denote the weights of the basic student. The consistency constraint is defined as:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),\mathcal{T}_x), \tag{15}$$

where $f(\theta,x)$ is the prediction from model $f(\theta)$ for input x. $\mathcal{T}_x$ is the consistency target of the teacher. $\mathcal{R}(\cdot,\cdot)$ measures the distance between two vectors and is usually set to Mean Squared Error (MSE) or KL-divergence. Different consistency regularization techniques vary in how they generate the target. There are several ways to boost the target $\mathcal{T}_x$ quality. One strategy is to select the perturbation rather than additive or multiplicative noise carefully. Another technique is to consider the teacher model carefully instead of replicating the student model.

Ladder Network. Ladder Network [56], [170] is the first successful attempt towards using a Teacher-Student model that is inspired by a deep denoising AutoEncoder. The structure of the Ladder Network is shown in Fig. 4(1). In Encoder, noise ζ is injected into all hidden layers as the corrupted feedforward path $x+\zeta \rightarrow \frac{\text{Encoder}}{f(\cdot)} \rightarrow \tilde{z}_1 \rightarrow \tilde{z}_2$ and shares the mappings $f(\cdot)$ with the clean encoder feedforward path $x \rightarrow \frac{\text{Encoder}}{f(\cdot)} \rightarrow z_1 \rightarrow z_2 \rightarrow y$. The decoder path $\tilde{z}_1 \rightarrow \tilde{z}_2 \rightarrow \frac{\text{Decoder}}{g(\cdot,\cdot)} \rightarrow \hat{z}_2 \rightarrow \hat{z}_1$ consists of the denoising functions $g(\cdot,\cdot)$ and the unsupervised denoising square error $\mathcal{R}$ on each layer consider as consistency loss between $\hat{z}_i$ and z_i. Through latent skip connections, the ladder network is differentiated from regular denoising AutoEncoder. This feature allows the higher layer features to focus on more abstract invariant features for the task. Formally, the ladder network unsupervised training loss $\mathcal{L}_u$ or the consistency loss is computed as the MSE between the activation of the clean encoder z_i and the reconstructed activations $\hat{z}_i$. Generally, $\mathcal{L}_u$ is

$$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x\right),g\left(f\left(\theta,x+\zeta\right)\right)\right). \tag{16}$$

Π Model. Unlike the perturbation used in Ladder Network, Π Model [57] is to create two random augmentations of a sample for both labeled and unlabeled data. Some techniques with non-deterministic behavior, such as randomized data augmentation, dropout, and random max-pooling, make an input sample pass through the network several times, leading to different predictions. The structure of the Π Model is shown in Fig. 4(2). In each epoch of the training process for Π Model, the same unlabeled sample propagates forward twice, while random perturbations are introduced by data augmentations and dropout. The forward propagation of the same sample may result in different predictions, and the Π Model expects the two predictions to be as consistent as possible. Therefore, it provides an

unsupervised consistency loss function,

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x,\zeta_1),f(\theta,x,\zeta_2)), \tag{17}$$

which minimizes the difference between the two predictions.

Temporal Ensembling. Temporal Ensembling [58] is similar to the Π Model, which forms a consensus prediction under different regularization and input augmentation conditions. The structure of Temporal Ensembling is shown in Fig. 4(3). It modifies the Π Model by leveraging the Exponential Moving Average (EMA) of past epochs predictions. In other words, while Π Model needs to forward a sample twice in each iteration, Temporal Ensembling reduces this computational overhead by using EMA to accumulate the predictions over epochs as $\mathcal{T}_x$. Specifically, the ensemble outputs Z_i is updated with the network outputs z_i after each training epoch, *i.e.*, $Z_i \leftarrow \alpha Z_i + (1-\alpha)\, z_i$, where α is a momentum term. During the training process, the Z can be considered to contain an average ensemble of $f(\cdot)$ outputs due to Dropout and stochastic augmentations. Thus, consistency loss is:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x,\zeta_1),\mathrm{EMA}(f(\theta,x,\zeta_2))). \tag{18}$$

Mean Teacher. Mean Teacher [59] averages model weights using EMA over training steps and tends to produce a more accurate model instead of directly using output predictions. The structure of Mean Teacher is shown in Fig. 4(4). Mean Teacher consists of two models called Student and Teacher. The student model is a regular model similar to the Π Model, and the teacher model has the same architecture as the student model with exponential moving averaging the student weights. Then Mean Teacher applied a consistency constraint between the two predictions of student and teacher:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x,\zeta),f(\mathrm{EMA}(\theta),x,\zeta')). \tag{19}$$

VAT. Virtual Adversarial Training [171] proposes the concept of adversarial attack for consistency regularization. The structure of VAT is shown in Fig. 4(5). This technique aims to generate an adversarial transformation of a sample, which can change the model prediction. Specifically, the adversarial training technique is used to find the optimal adversarial perturbation γ of a real input instance x such that $\gamma \leq \delta$. Afterward, the consistency constraint is applied between the model's output of the original input sample and the perturbed one, *i.e.*,

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),g(\theta,x+\gamma^{adv})), \tag{20}$$

where $\gamma^{adv} = \mathrm{argmax}_{\gamma;\|\gamma\|\leqslant\delta}\,\mathcal{R}(f(\theta,x),g(\theta,x+\gamma))$.

Dual Student. Dual Student [169] extends the Mean Teacher model by replacing the teacher with another student. The structure of Dual Student is shown in Fig. 4(6). The two students start from different initial states and are optimized through individual paths during training. The authors also defined a novel concept, "stable sample", along with a stabilization constraint to avoid the performance bottleneck produced by a coupled EMA Teacher-Student model. Hence, their weights may not be tightly coupled, and each learns its own knowledge. Formally, Dual Student checks whether x is a stable sample for student i:

$$\mathcal{C}_x^i = \left\{p_x^i = p_{\bar{x}}^i\right\}_1 \,\&\, \left(\left\{\mathcal{M}_x^i > \xi\right\}_1 \left\|\left\{\mathcal{M}_{\bar{x}}^i > \xi\right\}_1\right.\right), \tag{21}$$

where $\mathcal{M}_x^i = \left\|f\left(\theta^i,x\right)\right\|_\infty$, and the stabilization constraint:

$$\mathcal{L}_{sta}^i = \begin{cases} \left\{\varepsilon^i > \varepsilon^j\right\}_1 \mathcal{R}\left(f\left(\theta^i,x\right),f\left(\theta^j,x\right)\right) & \mathcal{C}^i=\mathcal{C}^j=1 \\ \mathcal{C}^i\mathcal{R}\left(f\left(\theta^i,x\right),f\left(\theta^j,x\right)\right) & \text{otherwise} \end{cases}. \tag{22}$$

SWA. Stochastic Weight Averaging (SWA) [172] improves generalization than conventional training. The aim is to average multiple points along the trajectory of stochastic gradient descent (SGD) with a cyclical learning rate and seek much flatter solutions than SGD. The consistency-based SWA [173] observes that SGD fails to converge on the consistency loss but continues to explore many solutions with high distances apart in predictions on the test data. The structure of SWA is shown in Fig. 4(7). The SWA procedure also approximates the Teacher-Student approach, such as Π Model and Mean Teacher with a single model. The authors propose fast-SWA, which adapts the SWA to increase the distance between the averaged weights by using a longer cyclical learning rate schedule and diversity of the corresponding predictions by averaging multiple network weights within each cycle. Generally, the consistency loss can be rewritten as follows:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),f(\mathrm{SWA}(\theta),x,\zeta)). \tag{23}$$

VAdD. In VAT, the adversarial perturbation is defined as an additive noise unit vector applied to the input or embedding spaces, which has improved the generalization performance of SSL. Similarly, Virtual Adversarial Dropout (VAdD) [174] also employs adversarial training in addition to the Π Model. The structure of VAdD is shown in Fig. 4(8). Following the design of Π Model, the consistency constraint of VAdD is computed from two different dropped networks: one dropped network uses a random dropout mask, and the other applies adversarial training to the optimized dropout network. Formally, $f(\theta,x,\epsilon)$ denotes an output of a neural network with a random dropout mask, and the consistency loss incorporated adversarial dropout is described as:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x,\epsilon^s),f(\theta,x,\epsilon^{adv})), \tag{24}$$

where $\epsilon^{adv} = \mathrm{argmax}_{\epsilon;\|\epsilon^s-\epsilon\|_2\leq\delta H}\,\mathcal{R}(f(\theta,x,\epsilon^s),f(\theta,x,\epsilon))$; $f(\theta,x,\epsilon^{adv})$ represents an adversarial target; ϵ^{adv} is an adversarial dropout mask; ϵ^s is a sampled random dropout mask instance; δ is a hyperparameter controlling the intensity of the noise, and H is the dropout layer dimension.

WCP. A novel regularization mechanism for training deep SSL by minimizing the Worse-case Perturbation (WCP) is presented by Zhang *et al.* [60]. The structure of WCP is shown in Fig. 4(9). WCP considers two forms of WCP regularizations – additive and DropConnect perturbations, which impose additive perturbation on network weights and make structural changes by dropping the network connections, respectively. Instead of generating an ensemble of randomly corrupted networks, the WCP suggests enhancing the most vulnerable part of a network by making the most resilient weights and connections against the worst-case perturbations. It enforces an additive noise on the model parameters ζ, along with a constraint on the norm of the noise. In this case, the WCP regularization becomes,

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),g(\theta+\zeta,x)). \tag{25}$$

TABLE 3
Summary of Consistency Regularization Methods.

Methods	Techniques	Transformations	Consistency Constraints
Ladder Network	Additional Gaussian Noise in every neural layer	input	$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),f(\theta,x+\zeta))$
Π Model	Different Stochastic Augmentations	input	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x,\zeta_1\right),f\left(\theta,x,\zeta_2\right)\right)$
Temporal Ensembling	Different Stochastic Augmentation and EMA the predictions	input, predictions	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x,\zeta_1\right),\text{EMA}\left(f\left(\theta,x,\zeta_2\right)\right)\right)$
Mean Teacher	Different Stochastic Augmentation and EMA the weights	input, weights	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x,\zeta\right),f\left(\text{EMA}\left(\theta\right),x,\zeta\right)\right)$
VAT	Adversarial perturbation	input	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x\right),f\left(\theta,x,\gamma^{adv}\right)\right)$
Dual Student	Stable sample and stabilization constraint	input, weights	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\text{STA}\left(\theta,x_i\right),\zeta_1\right),f\left(\text{STA}\left(\theta,x_j\right),\zeta_2\right)\right)$
SWA	Stochastic Weight Averaging	input, weights	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x\right),f\left(\text{SWA}\left(\theta\right),x,\zeta\right)\right)$
VAdD	Adversarial perturbation and Stochastic Augmentation (dropout mask)	input, weights	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x,\epsilon^s\right),f\left(\theta,x,\epsilon^{adv}\right)\right)$
UDA	RandAugment for image; Back-Translation for text)	input	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x\right),f\left(\theta,x,\zeta\right)\right)$
WCP	Additive perturbation on network weights, DropConnect perturbation for network structure	input, network structure	$\mathbb{E}_{x\in X}\mathcal{R}\left(f\left(\theta,x\right),g\left(\theta+\zeta,x\right)\right)$

The second perturbation is at the network structure level by DropConnect, which drops some network connections. Specifically, for parameters θ, the perturbed version is $(1-\alpha)\theta$, and the $\alpha = 1$ denotes a dropped connection while $\alpha = 0$ indicates an intact one. By applying the consistency constraint, we have

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),f((1-\alpha)\theta,x)). \tag{26}$$

UDA. UDA stands for Unsupervised Data Augmentation [175] for image classification and text classification. The structure of UDA is shown in Fig. 4(10). This method investigates the role of noise injection in consistency training and substitutes simple noise operations with high-quality data augmentation methods, such as AutoAugment [176], RandAugment [177] for images, and Back-Translation [178], [179] for text. Following the consistency regularization framework, the UDA [175] extends the advancement in supervised data augmentation to SSL. As discussed above, let $f(\theta,x,\zeta)$ be the augmentation transformation from which one can draw an augmented example (x,ζ) based on an original example x. The consistency loss is:

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta,x),f(\theta,x,\zeta)), \tag{27}$$

where ζ represents the data augmentation operator to create an augmented version of an input x.

Summary. The core idea of consistency regularization methods is that the output of the model remains unchanged under realistic perturbation. As shown in TABLE 3, Consistency constraints can be considered at three levels: input dataset, neural networks and training process. From the input dataset perspective, perturbations are usually added to the input examples: additive noise, random augmentation, or even adversarial training. We can drop some layers or connections for the networks, as WCP [60]. From the training process, we can use SWA to make the SGD fit the consistency training or EMA parameters of the model for some training epochs as new parameters.

5 Graph-based methods

Graph-based semi-supervised learning (GSSL) has always been a hot subject for research with a vast number of successful models because of its wide variety of applicability. The basic assumption in GSSL is that a graph can be extracted from the raw dataset where each node represents a training sample, and each edge denotes some similarity measurement of the node pair. Most of these approaches can be divided into two main categories: graph regularization and graph embedding. The former uses Laplacian regularization specifically, assuming that connected nodes with a strongly connected edge share probably the same labels, such as label propagation (LP) [34], Gaussian random fields (GRF) [180] and Local and Global Consistency (LGC) [21]. The principal goal of the latter is to encode the nodes as small-scale vectors representing their role and the structure information of their neighborhood.

For graph embedding methods, we have the formal definition for the embedding on the node level. Given the graph $\mathcal{G}(\mathcal{V},\mathcal{E})$, the node that has been embedded is a mapping result of $f_{\mathbf{z}}: v \rightarrow \mathbf{z}_v \in \mathbb{R}^d, \forall v \in \mathcal{V}$ such that $d \ll |\mathcal{V}|$, and the $f_{\mathbf{z}}$ function retains some of the measurement of proximity, defined in the graph $\mathcal{G}$.

The unified form of the loss function is shown as Eq. (28) for graph embedding methods.

$$\mathcal{L}(f_{\mathbf{z}}) = \sum_{(x,y)\in X_l}(x,y,f_{\mathbf{z}}) + \alpha\sum_{x\in X}\mathcal{R}(x,f_{\mathbf{z}}), \tag{28}$$

where $f_{\mathbf{z}}$ is the embedding function.

Besides, we can divide graph embedding methods into shallow embedding and deep embedding based on the use or not use of deep learning techniques. The encoder's function, as a simple lookup function based on node ID, is built with shallow embedding methods including DeepWalk [181], LINE [182] and node2vec [183], whereas the deep embedding encoder is far more complicated, and the deep learning frameworks have to make full use of node attributes.

As deep learning progresses, recent GSSL research has switched from shallow embedding methods to deep embedding methods in which the $f_{\mathbf{z}}$ embedding term in the Eq. (28) is focused on deep learning models. Two classes can be identified among all deep embedding methods: AutoEncoder-based methods and GNN-based methods.

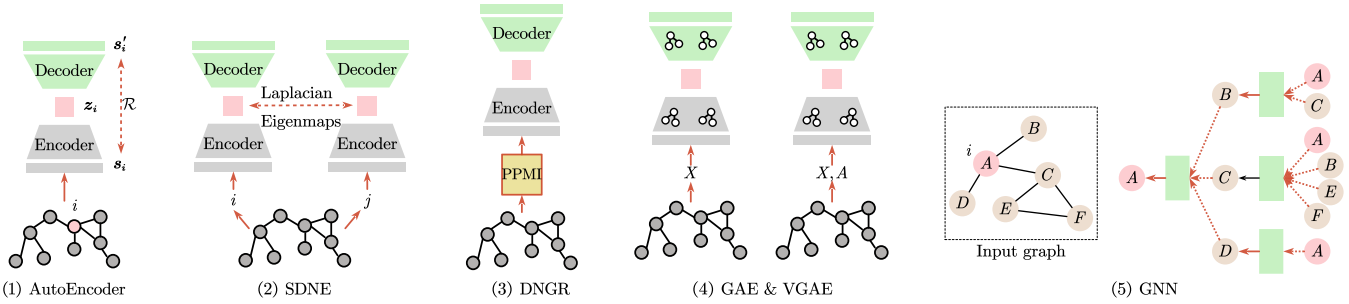

Fig. 5. A glimpse of the diverse range of architectures used for graph-based semi-supervised methods. Specifically, in figure (3), "PPMI" is short for positive pointwise mutual information. In figure (4), A denotes the adjacency matrix, and in figure (5), pink A represents node A.

5.1 AutoEncoder-based methods

AutoEncoder-based approaches also differ as a result of using a unary decoder rather than a pairwise method. More precisely, every node, i, is correlated to a neighborhood vector $\mathbf{s}_i \in \mathbb{R}^{|\mathcal{V}|}$. The $\mathbf{s}_i$ vector contains i's similarity with all other graph nodes and acts as a high dimensional vector representation of i in the neighborhood. The aim of the auto-encoding technique is to embed nodes using hidden embedding vectors like $\mathbf{s}_i$ in so as to reconstruct the original information from such embeddings (Fig. 5(1)):

$$\text{Dec}\left(\text{Enc}\left(\mathbf{s}_i\right)\right) = \text{Dec}\left(\mathbf{z}_i\right) \approx \mathbf{s}_i. \tag{29}$$

In other words, the loss for these methods takes the following form:

$$\mathcal{L} = \sum_{i \in V} \left\| \text{Dec}\left(\mathbf{z}_i\right) - \mathbf{s}_i \right\|_2^2. \tag{30}$$

SDNE. Structural deep network embedding (SDNE) is developed by Wang *et al.* [184] by using deep autoencoders to maintain the first and second-order network proximities. This is managed by optimizing the two proximities simultaneously. The process utilizes highly nonlinear functions to obtain its embedding. This framework consists of two parts: unsupervised and supervised. The first is an autoencoder to identify an embedding for a node to rebuild the neighborhood. The second is based on Laplacian Eigenmaps [185], which imposes a penalty when related vertices are distant from each other.

DNGR. (DNGR) [186] combines random surfing with autoencoders. The model includes three components: random surfing, positive pointwise mutual information (PPMI) calculation, and stacked denoising autoencoders. The random surfing design is used to create a stochastic matrix equivalent to the similarity measure matrix in HOPE [187] on the input graph. The matrix is transformed into a PPMI matrix and fed into a stacked denoising autoencoder to obtain the embedding. The PPMI matrix input ensures that the autoencoder model captures a higher proximity order. The use of stacked denoising autoencoders also helps make the model robust in the event of noise in the graph, as well as in seizing the internal structure needed for downstream tasks.

GAE & VGAE. Both MLP-based and RNN-based strategies only consider the contextual information and ignore the feature information of the nodes. To encode both, GAE [188] uses GCN [189]. The encoder is in the form,

$$\text{Enc}(A, X) = \text{GraphConv}\left(\sigma(\text{GraphConv}(A, X))\right), \tag{31}$$

where $\text{GraphConv}(\cdot)$ is a graph convolutional layer defined in [189], $\sigma(\cdot)$ is the activation function and A, X is adjacency matrix and attribute matrix respectively. The decoder of GAE is defined as

$$\text{Dec}\left(\mathbf{z}_u, \mathbf{z}_v\right) = \mathbf{z}_u^T \mathbf{z}_v. \tag{32}$$

Variational GAE (VGAE) [188] learns about the distribution of the data in which the variation lower bound $\mathcal{L}$ is optimized directly by reconstructing the adjacency matrix.

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{Z}|X,A)}[\log p(A \mid \mathbf{Z})] - \text{KL}[q(\mathbf{Z} \mid X, A) \| p(\mathbf{Z})], \tag{33}$$

where $\text{KL}[q(\cdot) \| p(\cdot)]$ is the Kullback-Leibler divergence between $q(\cdot)$ and $p(\cdot)$. Moreover, we have

$$q(\mathbf{Z} \mid X, A) = \prod_{i=1}^{N} \mathcal{N}\left(\mathbf{z}_i \mid \mu_i, \text{diag}\left(\sigma_i^2\right)\right), \tag{34}$$

and

$$p(A \mid \mathbf{Z}) = \prod_{i=1}^{N} A_{ij} \sigma\left(\mathbf{z}_i^\top \mathbf{z}_j\right) + (1 - A_{ij})\left(1 - \sigma\left(\mathbf{z}_i^\top \mathbf{z}_j\right)\right). \tag{35}$$

Summary. It should be noted from Eq. (29) that the encoder unit does depend on the specific $\mathbf{s}_i$ vector, which gives the crucial information relating to the local community structure of v_i. TABLE 4 sums up the main components of these methods, and their architectures are compared as Fig. 5.

5.2 GNN-based methods

Several updated, deep embedding strategies are designed to resolve major autoencoder-based method disadvantages by building certain specific functions that rely on the local community of the node but not necessarily the whole graph (Fig. 5(5)). The GNN, which is widely used in state-of-the-art deep embedding approaches, can be regarded as a general guideline for the definition of deep neural networks on graphs.

Like other deep node-level embedding methods, a classifier is trained to predict class labels for the labeled nodes. Then it can be applied to the unlabeled nodes based on the final, hidden state of the GNN-based model. Since GNN

TABLE 4
Summary of AutoEncoder-based Deep Graph Embedding Methods

Method	Encoder	Decoder	Similarity Measure	Loss Function	Time Complexity
SDNE [184]	MLP	MLP	$\mathbf{s}_u$	$\sum_{u\in\mathcal{V}} \|\text{Dec}(\mathbf{z}_u) - \mathbf{s}_u\|_2^2$	$O(\|\mathcal{V}\|\mathcal{E}\|)$
DNGR [186]	MLP	MLP	$\mathbf{s}_u$	$\sum_{u\in\mathcal{V}} \|\text{Dec}(\mathbf{z}_u) - \mathbf{s}_u\|_2^2$	$O(\|\mathcal{V}\|^2)$
GAE [188]	GCN	$\mathbf{z}_u^\top \mathbf{z}_v$	A_{uv}	$\sum_{u\in\mathcal{V}} \|\text{Dec}(\mathbf{z}_u) - A_u\|_2^2$	$O(\|\mathcal{V}\|\mathcal{E}\|)$
VGAE [188]	GCN	$\mathbf{z}_u^\top \mathbf{z}_v$	A_{uv}	$\mathbb{E}_{q(\mathbf{Z}\|X,A)}[\log p(A \mid \mathbf{Z})] - \text{KL}[q(\mathbf{Z} \mid X, A)\|p(\mathbf{Z})]$	$O(\|\mathcal{V}\|\mathcal{E}\|)$

consists of two primary operations: the aggregate operation and the update operation, the basic GNN is provided, and then some popular GNN extensions are reviewed with a view to enhancing each process, respectively.

Basic GNN. As Gilmer *et al.* [190] point out, the critical aspect of a basic GNN is that the benefit of *neural message passing* is to exchange and update messages between each node pair by using neural networks.

More specifically, a hidden embedding $\mathbf{h}_u^{(k)}$ in each neural message passing through the GNN basic iteration is updated according to message or information from the neighborhood within $\mathcal{N}(u)$ according to each node u. This general message can be expressed according to the update rule:

$$\begin{aligned}\mathbf{h}_u^{(k+1)} &= \text{Update}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{Aggregate}^{(k)}\left(\left\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right)\right) \\ &= \text{Update}^{(k)}\left(\mathbf{h}_u^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right),\end{aligned} \tag{36}$$

where

$$\mathbf{m}_{\mathcal{N}(u)}^{(k)} = \text{Aggregate}^{(k)}\left(\left\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right). \tag{37}$$

It is worth noting that the functions **Update** and **Aggregate** must generally be differentiable in Eq. (36). The new state is generated in accordance with Eq. (36) when the neighborhood message is combined with the previous hidden embedding state. After a number of iterations, the last hidden embedding state converges so that each node's final status is created as the output. Formally, we have, $\mathbf{z}_u = \mathbf{h}_u^{(K)}, \forall u \in \mathcal{V}$.

The basic GNN model is introduced before reviewing many other GNN based methods designed to perform SSL tasks. The basic version of GNN aims to simplify the original GNN model, proposed by Scarselli *et al.* [191].

The basic GNN message passing is defined as:

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)}\sum_{v\in\mathcal{N}(u)}\mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right), \tag{38}$$

where $\mathbf{W}_{\text{self}}^{(k)}, \mathbf{W}_{\text{neigh}}^{(k)}$ are trainable parameters and σ is the activation function. In principle, the messages from the neighbors are summarized first. Then, the neighborhood information and the previously hidden node results are integrated by an essential linear combination. Finally, the joint information uses a nonlinear activation function. It is worth noting that the GNN layer can be easily stacked together following Eq. (38). The last layer's output in the GNN model is regarded as the final node embedding result to train a classifier for the downstream SSL tasks.

As previously mentioned, GNN models have all sorts of variants that try to some degree boost their efficiency and robustness. All of them, though, obey the Eq. (36) neural message passing structure, regardless of the GNN version explored previously.

GCN. As mentioned above, the most simple neighborhood aggregation operation only calculates the sum of the neighborhood encoding states. The critical issue with this approach is that nodes with a large degree appear to derive a better benefit from more neighbors compared to those with a lower number of neighbors.

One typical and straightforward approach to this problem is to normalize the aggregation process depending on the central node degree. The most popular method is to use the following symmetric normalization Eq. (39) employed by Kipf *et al.* [189] in the graph convolutional network (GCN) model as Eq. (39).

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v\in\mathcal{N}(u)} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}} \tag{39}$$

GCN fully uses the uniform neighborhood grouping technique. Consequently, the GCN model describes the update function as Eq. (40). As it is indirectly specified in the update function, no aggregation operation is defined.

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}^{(k)}\sum_{v\in\mathcal{N}(u)\cup\{u\}} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}\right) \tag{40}$$

A vast range of GCN variants is available to boost SSL performance from various aspects. Li *et al.* [192] were the first to have a detailed insight into the performance and lack of GCN in SSL tasks. Subsequently, GCN extensions for SSL started to propagate [193] [194] [195] [196] [197].

GAT. In addition to more general ways of set aggregation, another common approach for improving the aggregation layer of GNNs is to introduce certain attention mechanisms [198]. The basic theory is to give each neighbor a weight or value of significance which is used to weight the influence of this neighbor during the aggregation process. The first GNN to use this focus was Cucurull *et al.*'s Graph Attention Network (GAT), which uses attention weights to describe a weighted neighboring amount:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v\in\mathcal{N}(u)} \alpha_{u,v}\mathbf{h}_v, \tag{41}$$

where $\alpha_{u,v}$ denotes the attention on neighbor $v \in \mathcal{N}(u)$ when we are aggregating information at node u. In the original GAT paper, the attention weights are defined as

$$\alpha_{u,v} = \frac{\exp\left(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v]\right)}{\sum_{v'\in\mathcal{N}(u)}\exp\left(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}]\right)}, \tag{42}$$

where $\mathbf{a}$ is a trainable attention vector, $\mathbf{W}$ is a trainable matrix, and denotes the concatenation operation.

GraphSAGE. Over-smoothing is an obvious concern for GNN. Over-smoothing after several message iterations is almost unavoidable as the node-specific information is "washed away." The use of vector concatenations or skip connections, which both preserve information directly from the previous rounds of the update, is one fair way to lessen this concern. For general purposes, $\text{Update}_{\text{base}}$ denotes a basic update rule.

One of the simplest updates in GraphSage [199] for skip connection uses a concatenation vector to contain more node-level information during a messages passage process:

$$\text{Update}\left(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}}(u)\right) = \left[\text{Update}_{\text{base}}\left(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}\right) \oplus \mathbf{h}_u\right], \tag{43}$$

where the output from the basic update function is concatenated with the previous layer representation of the node. The critical observation is that this designed model is encouraged to detach information during the message passing operation.

GGNN. Parallel to the above work, researchers also are motivated by the approaches taken by recurrent neural networks (RNNs) to improve stability. One way to view the GNN message propagation algorithm is to gather an observation from neighbors of the aggregation process and then change each node's hidden state. In this respect, specific approaches can be used explicitly based on observations to check the hidden states of RNN architectures.

For example, one of the earliest GNN variants which put this idea into practice is proposed by Li *et al.* [200], in which the update operation is defined as Eq. (44)

$$\mathbf{h}_u^{(k)} = \text{GRU}\left(\mathbf{h}_u^{(k-1)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right), \tag{44}$$

where GRU is a gating mechanism function in recurrent neural networks, introduced by Kyunghyun Cho *et al.* [201]. Another related approach would be similar improvements based on the LSTM architecture [202].

Summary. The main point of graph-based models for DSSL is to perform label inference on a constructed similarity graph so that the label information can be propagated from the labeled samples to the unlabeled ones by incorporating both the topological and feature knowledge. Moreover, the involvement of deep learning models in GSSL helps generate more discriminative embedding representations that are beneficial for the downstream SSL tasks, thanks to the more complex encoder functions.

6 Pseudo-Labeling Methods

6.1 Disagreement-based models

The idea of disagreement-based SSL is to train multiple learners for the task and exploit the disagreement during the learning process [203]. In such model designs, two or three different networks are trained simultaneously and label unlabeled samples for each other. Disagreement-based methods differ in whether the data has multiple views *i.e.*, Co-training [22], [204] for multiview data or Tri-Net [205] for single-view data.

Deep co-training. Co-training [22] framework assumes each data x in the dataset has two different and complementary views, and each view is sufficient for training a good classifier. Because of this assumption, Co-training learns two different classifiers on these two views (see Fig. 6(1)). Then the two classifiers are applied to predict each view's unlabeled data and label the most confident candidates for the other model. This procedure is iteratively repeated till unlabeled data are exhausted, or some condition is met (such as the maximum number of iterations is reached). Let v_1 and v_2 as two different views of data such that $x = (v_1, v_2)$. Co-training assumes that $\mathcal{C}_1$ as the classifier trained on View-1 v_1 and $\mathcal{C}_2$ as the classifier trained on View-2 v_2 have consistent predictions on $\mathcal{X}$. In the objective function, the Co-training assumption can be model as:

$$\mathcal{L}_{ct} = H(\frac{1}{2}(\mathcal{C}_1(v_1) + \mathcal{C}_2(v_2))) - \frac{1}{2}(H(\mathcal{C}_1(v_1)) + H(\mathcal{C}_2(v_2))), \tag{45}$$

where $H(\cdot)$ denotes the entropy, the Co-training assumption is formulated as $\mathcal{C}(x) = \mathcal{C}_1(v_1) = \mathcal{C}_2(v_2), \forall x = (v_1, v_2) \sim \mathcal{X}$. On the labeled dataset X_L, the supervised loss function can be the standard cross-entropy loss

$$\mathcal{L}_s = H(y, \mathcal{C}_1(v_1)) + H(y, \mathcal{C}_2(v_2)), \tag{46}$$

where $H(p, q)$ is the cross-entropy between distribution p and q.

The key to the success of Co-training is that the two views are different and complementary. However, the loss function $\mathcal{L}_{ct}$ and $\mathcal{L}_s$ only ensure the model tends to be consistent for the predictions on the dataset. To address this problem, [204] forces to add the View Difference Constraint to the previous Co-training model, and formulated as:

$$\exists \mathcal{X}' : \mathcal{C}_1(v_1) \neq \mathcal{C}_2(v_2), \forall x = (v_1, v_2) \sim \mathcal{X}', \tag{47}$$

where $\mathcal{X}'$ denotes the adversarial examples of $\mathcal{X}$, thus $\mathcal{X}' \cap \mathcal{X} = \emptyset$. In the loss function, the View Difference Constraint can be model by minimizing the cross-entropy between $\mathcal{C}_2(x)$ and $\mathcal{C}_1(g_2(x))$, where $g(\cdot)$ denotes the adversarial examples generated by the generative model. Then, this part loss function is:

$$\mathcal{L}_{dif}(x) = H(\mathcal{C}_1(x), \mathcal{C}_2(g_1(x))) + H(\mathcal{C}_2(x), p_1(g_2(x))). \tag{48}$$

In Deep Co-training [204], the objective function can be formed as:

$$\mathcal{L} = \mathbb{E}_{(x,y)\in X_L}\mathcal{L}_s(x, y) + \alpha\mathbb{E}_{x\in X_U}\mathcal{L}_{ct}(x) + \beta\mathbb{E}_{x\in X}\mathcal{L}_{dif}(x), \tag{49}$$

where α and β are linear hyper-parameters.

Some other research works also explore to apply co-training into neural network model training. For example, [206] treats the RGB and depth of an image as two independent views for object recognition. Then, co-training is performed to train two networks on the two views. Next, a fusion layer is added to combine the two-stream networks for recognition, and the overall model is jointly trained. Besides, in sentiment classification, [207] considers the original review and the automatically constructed anonymous review as two opposite sides of one review and then apply the co-training algorithm. One crucial property of [207] is that two views are opposing and therefore associated with opposite class labels.

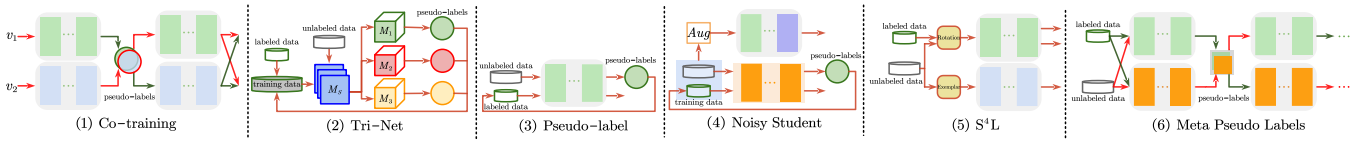

Fig. 6. A glimpse of the diverse range of architectures used for pseudo-label semi-supervised methods. The same color and structure have the same meaning as shown in Figure 4. M_s denotes shared module, M_1, M_2 and M_3 are three different modules in Tri-Net. "Rotation " and "Exemplar" represent S^4L-Rotation and S^4L-Exemplar, respectively.

Tri-Net. Tri-net [205], a deep learning-based method inspired by the tri-training [208]. The tri-training learns three classifiers from three different training sets, which are obtained by utilizing bootstrap sampling. The framework (as shown in Fig. 6(2)) of tri-net can be intuitively described as follows. Output smearing [209] is used to add random noise to the labeled sample to generate different training sets and help learn three initial modules. The three models then predict the pseudo-label for unlabeled data. With the predictions of two modules for unlabeled instances consistent, the pseudo-label is considered to be confident and stable. The labeled sample is added to the training set of the third module, and then the third module is fine-tuned on the augmented training set. During the augmentation process, the three modules become more and more similar, so the three modules are fine-tuned on the training sets respectively to ensure diversity. Formally, the output smearing is used to construct three different training sets $\{\mathcal{L}_{os}^j = (x_i, \hat{y}_i^j), j = 1, 2, 3\}$ from the initial labeled set X_L. Then tri-net can be initialized by minimizing the sum of standard softmax cross-entropy loss function from the three training sets,

$$\begin{aligned}\mathcal{L} =& \frac{1}{L}\sum_{i=1}^{L}\{\mathcal{L}_y(M_1(M_S(x_i)), \hat{y}_i^1) + \mathcal{L}_y(M_2(M_S(x_i)), \hat{y}_i^2) \\ &+\mathcal{L}_y(M_3(M_S(x_i)), \hat{y}_i^3)\}, \end{aligned} \tag{50}$$

where $\mathcal{L}_y$ is the standard softmax cross-entropy loss function; M_S denote a shared module, and M_1, M_2, M_3 is the three different modules; $M_j(M_S(x_i)), j = 1, 2, 3$ denotes the outputs of the shared features generated by M_S. In the whole procedure, the unlabeled sample can be pseudo-labeled by the maximum posterior probability,

$$\begin{aligned} y = & \underset{k\in\{1,2,\ldots,K\}}{\operatorname{argmax}} \{p(M_1(M_S(x)) = k|x) + \\ & p(M_2(M_S(x)) = k|x) + p(M_3(M_S(x)) = k|x)\}. \end{aligned}$$

Summary. The disagreement-based SSL methods exploit the unlabeled data by training multiple learners, and the "disagreement" among these learners is crucial. When the data has two sufficient redundancy and conditional independence views, Deep Co-training [204] improves the disagreement by designing a View Difference Constraint. Tri-Net [205] obtains three labeled datasets by bootstrap sampling and trains three different learners. These methods in this category are less affected by model assumptions, non-convexity of the loss function and the scalability of the learning algorithms.

6.2 Self-training models

Self-training algorithm leverages the model's own confident predictions to produce the pseudo labels for unlabeled data. In other words, it can add more training data by using existing labeled data to predict the labels of unlabeled data. Because of its simplicity and generality, self-training is successfully applied in various tasks such as Named Entity Recognition [210], contour detection [211], machine translation [212], and object detection [213]. Specifically, [210] chooses the most confident named entity recognition predictions of the unlabeled data as the additional targets to boost the performance. [212] first trains the neural translation model on the parallel corpus and then uses the learned model to translate a monolingual corpus, wherein the monolingual corpus and its translations constitute an additional pseudo-parallel corpus.

EntMin. Entropy Minimization (EntMin) [214] is a method of entropy regularization, which can be used to realize SSL by encouraging the model to make low-entropy predictions for unlabeled data and then using the unlabeled data in a standard supervised learning setting. In theory, entropy minimization can prevent the decision boundary from passing through a high-density data points region, otherwise it would be forced to produce low-confidence predictions for unlabeled data.

Pseudo-label. Pseudo-label [215] proposes a simple and efficient formulation of training neural networks in a semi-supervised fashion, in which the network is trained in a supervised way with labeled and unlabeled data simultaneously. As illustrated in Fig. 6(3), the model is trained on labeled data in a usual supervised manner with a cross-entropy loss. For unlabeled data, the same model is used to get predictions for a batch of unlabeled samples. The maximum confidence prediction is called a pseudo-label, which has the maximum predicted probability.

That is, the pseudo-label model trains a neural network with the loss function $\mathcal{L}$, where:

$$\mathcal{L} = \frac{1}{n}\sum_{m=1}^{n}\sum_{i=1}^{K}\mathcal{R}(y_i^m, f_i^m) + \alpha(t)\frac{1}{n'}\sum_{m=1}^{n'}\sum_{i=1}^{K}\mathcal{R}(y_i^{'m}, f_i^{'m}), \tag{51}$$

where n is the number of mini-batch in labeled data for SGD, n' for unlabeled data, f_i^m is the output units of m's sample in labeled data, y_i^m is the label of that, $y_i^{'m}$ for unlabeled data, $y_i^{'m}$ is the pseudo-label of that for unlabeled data, $\alpha(t)$ is a coefficient balancing the supervised and unsupervised loss terms.

Noisy Student. Noisy Student [216] proposes a semi-supervised method inspired by knowledge distillation [217] with equal-or-larger student models. The framework is shown in Fig. 6(4). The teacher EfficientNet [218] model is first trained on labeled images to generate pseudo labels for unlabeled examples. Then a larger EfficientNet model as a student is trained on the combination of labeled and

pseudo-labeled examples. These combined instances are augmented using data augmentation techniques such as RandAugment [219], and model noise such as Dropout and stochastic depth are also incorporated in the student model during training. After a few iterations of this algorithm, the student model becomes the new teacher to relabel the unlabeled data and this process is repeated.

S^4L. Self-supervised Semi-supervised Learning (S^4L) [220] tackles the problem of SSL by employing self-supervised learning [221] techniques to learn useful representations from the image databases. The architecture of S^4L is shown in Fig. 6(5). The conspicuous self-supervised techniques are predicting image rotation [222] and exemplar [223], [224]. Predicting image rotation is a pretext task that anticipates an angle of the rotation transformation applied to an input example. In S^4L, there are four rotation degrees $\{0°, 90°, 180°, 270°\}$ to rotate input images. The S^4L-Rotation loss is the cross-entropy loss on outputs predicted by those rotated images. S^4L-Exemplar introduces an exemplar loss which encourages the model to learn a representation that is invariant to heavy image augmentations. Specifically, eight different instances of each image are produced by inception cropping [225], random horizontal mirroring, and HSV-space color randomization as in [223]. Following [221], the loss term on unsupervised images uses the batch hard triplet loss [226] with a soft margin.

MPL. In the SSL, the target distributions are often generated on unlabeled data by a shaped teacher model trained on labeled data. The constructions for target distributions are heuristics that are designed prior to training, and cannot adapt to the learning state of the network training. Meta Pseudo Labels (MPL) [227] designs a teacher model that assigns distributions to input examples to train the student model. Throughout the course of the student's training, the teacher observes the student's performance on a held-out validation set, and learns to generate target distributions so that if the student learns from such distributions, the student will achieve good validation performance. The training procedure of MPL involves two alternating processes. As shown in Fig. 6(6), the teacher $g_\phi(\cdot)$ produces the conditional class distribution $g_\phi(x)$ to train the student. The pair $(x, g_\phi(x))$ is then fed into the student network $f_\theta(\cdot)$ to update its parameters θ from the cross-entropy loss. After the student network updates its parameters, the model evaluates the new parameters θ' based on the samples from the held-out validation dataset. Since the new parameters of the student depend on the teacher, the dependency allows us to compute the gradient of the loss to update the teacher's parameters.

Summary. In general, self-training is a way to get more training data by a series of operations to get pseudo-labels of unlabeled data. Both EntMin [214] and Pseudo-label [215] use entropy minimization to get the pseudo-label with the highest confidence as the ground truth for unlabeled data. Noisy Student [216] utilizes a variety of techniques when training the student network, such as data augmentation, dropout and stochastic depth. S^4L [220] not only uses data augmentation techniques, but also adds another 4-category task to improve model performance. MPL [227] modifies Pseudo-label [215] by deriving the teacher network's update rule from the feedback of the student network. Emerging techniques (*e.g.*, rich data augmentation strategies, meta-learning, self-supervised learning) and network architecture (*e.g.*, EfficientNet [218]) provide powerful support for the development of self-training methods.

7 Hybrid methods

Hybrid methods combine ideas from the above-mentioned methods such as pseudo-label, consistency regularization and entropy minimization for performance improvement. Moreover, a learning principle, namely Mixup [228], is introduced in those hybrid methods. It can be considered as a simple, data-agnostic data augmentation approach, a convex combination of paired samples and their respective labels. Formally, Mixup constructs virtual training examples,

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \tilde{y} = \lambda y_i + (1-\lambda)y_j, \tag{52}$$

where (x_i, y_i) and (x_j, y_j) are two instances from the training data, and $\lambda \in [0, 1]$. Therefore, Mixup extends the training data set by a hard constraint that linear interpolations of samples should lead to the linear interpolations of the corresponding labels.

ICT. Interpolation Consistency Training (ICT) [61] regularizes SSL by encouraging the prediction at an interpolation of two unlabeled examples to be consistent with the interpolation of the predictions at those points. The architecture is shown in Fig.7(1). The low-density separation assumption inspires this method, and Mixup can achieve broad margin decision boundaries. It is done by training the model $f(\theta)$ to predict $\text{Mix}_\lambda(\hat{y}_j, \hat{y}_k)$ at location $\text{Mix}_\lambda(x_j, x_k)$ with the Mixup operation: $\text{Mix}_\lambda(a, b) = \lambda a + (1-\lambda)b$. In semi-supervised settings, ICT extends Mixup by training the model $f(\theta, x)$ to predict the "fake label" $\text{Mix}_\lambda(f(\theta, x_j), f(\theta, x_k))$ at location $\text{Mix}_\lambda(x_j, x_k)$. Moreover, the model f_θ predict the fake label $\text{Mix}_\lambda(f_{\theta'}(x_i), f_{\theta'}(x_j))$ at location $\text{Mix}_\lambda(x_i, x_j)$, where θ' is a moving average of θ, like [59], for a more conservation consistent regularization. Thus, the ICT term is

$$\mathbb{E}_{x\in X}\mathcal{R}(f(\theta, \text{Mix}_\lambda(x_i, x_j)), \text{Mix}_\lambda(f(\theta', x_i), f(\theta', x_j)). \tag{53}$$

MixMatch. MixMatch [62] combines consistency regularization and entropy minimization in a unified loss function. This model operates by producing pseudo-labels for each unlabeled instance and then training the original labeled data with the pseudo-labels for the unlabeled data using fully-supervised techniques. The main aim of this algorithm is to create the collections X'_L and X'_U, which are made up of augmented labeled and unlabeled samples that were generated using Mixup. Formally, MixMatch produces an augmentation of each labeled instance (x_i, y_i) and K weakly augmented version of each unlabeled instance (x_j) with $k \in \{1, \ldots, K\}$. Then, it generates a pseudo-label $\bar{y}_j$ for each x_j by computing the average prediction across the K augmentations. The pseudo-label distribution is then sharpened by adjusting temperature scaling to get the final pseudo-label $\tilde{y}_j$. After the data augmentation, the batches of augmented labeled examples and unlabeled with pseudo-label examples are combined, then the whole group is shuffled. This group is divided into two parts: the first L samples were taken as $\mathcal{W}_L$, and the remaining taken

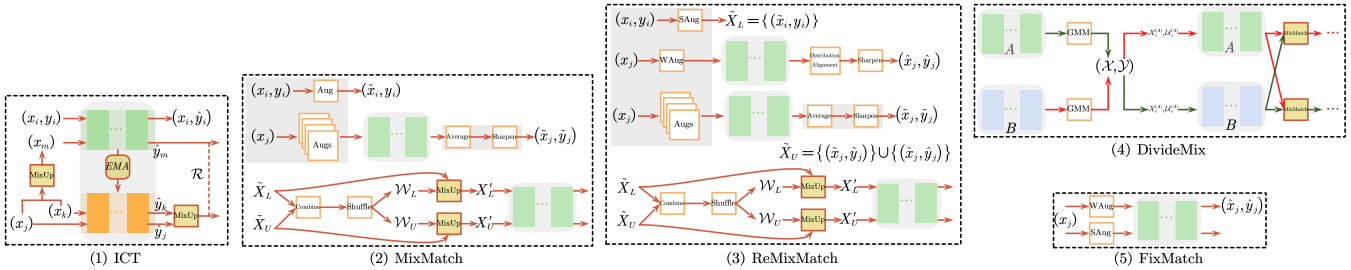

Fig. 7. A glimpse of the diverse range of architectures used for hybrid semi-supervised methods. "Mixup" is Mixup operator [228]. "MixMatch" is MixMatch [62] in figure (2). "GMM" is short for Gaussian Mixture Model. "SAug" and "WAug" represent Strong Augmentation and Weak Augmentation, respectively.

as $\mathcal{W}_U$. The group $\mathcal{W}_L$ and the augmented labeled batch $\tilde{X}_L$ are fed into the Mixup algorithm to compute examples (x', y') where $x' = \lambda x_1 + (1-\lambda)x_2$ for $\lambda \sim \text{Beta}(\alpha, \alpha)$. Similarly, Mixup is applied between the remaining $\mathcal{W}_U$ and the augmented unlabeled group $\tilde{X}_U$. MixMatch conducts traditional fully-supervised training with a standard cross-entropy loss for a supervised dataset and a mean square error for unlabeled data given these mixed-up samples.

ReMixMatch. ReMixMatch [63] extends MixMatch [62] by introducing distribution alignment and augmentation anchoring. Distribution alignment encourages the marginal distribution of aggregated class predictions on unlabeled data close to the marginal distribution of ground-truth labels. Augmentation anchoring replaces the consistency regularization component of MixMatch. This technique generates multiple strongly augmented versions of input and encourages each output to be close to predicting a weakly-augmented variant of the same input. A variant of AutoAugment [176] dubbed "CTAugment" is also proposed to produce strong augmentations, which learns the augmentation policy alongside the model training. As the procedure of ReMixmatch presented in Fig. 7(3), an "anchor" is generated by applying weak augmentation to a given unlabeled example and then K strongly-augmented versions of the same unlabeled example using CTAugment.

DivideMix. DivideMix [64] presents a new SSL framework to handle the problem of learning with noisy labels. As shown in Fig. 7(4), DivideMix proposes co-divide, a process that trains two networks simultaneously. For each network, a dynamic Gauss Mixed Model (GMM) is fitted on the loss distribution of each sample to divide the training set into labeled data and unlabeled data. The separated data sets are then used to train the next epoch's networks. In the follow-up SSL process, co-refinement and co-guessing are used to improve MixMatch [62] and solve the problem of learning with noisy labels.

FixMatch. FixMatch [229] combines consistency regularization and pseudo-labeling while vastly simplifying the overall method. The key innovation comes from the combination of these two ingredients, and the use of a separate weak and strong augmentation in the consistency regularization approach. Given an instance, only when the model predicts a high-confidence label can the predicted pseudo-label be identified as ground-truth. As shown in Fig.7(5), given an instance x_j, FixMatch firstly generates pseudo-label $\hat{y}_j$ for weakly-augmented unlabeled instances $\hat{x}_j$. Then, the model trained on the weakly-augmented samples is used to predict pseudo-label in the strongly-augmented version of x_j. In FixMatch, weak augmentation is a standard flip-and-shift augmentation strategy, randomly flipping images horizontally with a probability. For strong augmentation, there are two approaches which are based on [176], *i.e.*, RandAugment [219] and CTAugment [63]. Moreover, Cutout [230] is followed by either of these strategies.

TABLE 5
Summary of Input Augmentations and Neural Network Transformations

	Techniques	Methods
Input augmentations	Additional Noise	Ladder Network [56], WCP [60]
	Stochastic Augmentation	Π Model [57], Temporal Ensembling [58], Mean Teacher [59], Dual Student [169], MixMatch [62], ReMixMatch [63], FixMatch [229]
	Adversarial perturbation	VAT [171], VAdD [174]
	AutoAugment	UDA [175], Noisy Student [216]
	RandAugment	FixMatch [229]
	CTAugment	ReMixMatch [63], FixMatch [229]
	Mixup	ICT [61], MixMatch [62], ReMixMatch [63], DivideMix [64]
Neural network transformations	Dropout	Π Model [57], Temporal Ensembling [58], Mean Teacher [59], Dual Student [169], VAdD [174],Noisy Student [216]
	EMA	Mean Teacher [59], ICT [61]
	SWA	SWA [173]
	Stochastic depth	Noisy Student [216]
	DropConnect	WCP [60]

Summary. As discussed above, the hybrid methods unit the most successful approaches in SSL, such as pseudo-labeling, entropy minimization and consistency regularization, and adapt them to achieve state-of-the-art performance. In TABLE 5, we summarize some techniques that can be used in consistency training to improve model performance.

8 Challenges and Future Directions

Although exceptional performance and achieved promising DSSL progress, there are still several open research questions for future work. We outline some of these issues and future directions below.

Theoretical analysis. Existing semi-supervised approaches predominantly use unlabeled samples to generate a constraint and then update the model with labeled data and constraints. Generally, there is a single weight to balance the supervised and unsupervised loss, which means that all unlabeled instances are equally weighted. However, not all unlabeled data is equally appropriate for the model in practice. To counter this issue, [231] considers how to use a different weight for every unlabeled example. An algorithm adjusts those weights based on the influence function, which evaluates the dependence of the model on one training example.

Incorporation of domain knowledge. Most of the SSL approaches listed above can obtain satisfactory results only in ideal situations in which the training dataset meets the designed assumptions and contains sufficient information to learn an insightful learning system. However, in practice, the distribution of the dataset is unknown and does not necessarily meet these ideal conditions. When the distributions of labeled and unlabeled data do not belong to the same one or the model assumptions are incorrect, the more unlabeled data is utilized, the worse the performance will be. Therefore, we can attempt to incorporate richer and more reliable domain knowledge into the model to mitigate the degradation performance. Recent works focusing on this direction have proposed [232], [233], [234], [235], [236] for DSSL.

Learning with noisy labels. In this survey, models discussed typically consider the labeled data is generally accurate to learn a standard cross-entropy loss function. An interesting consideration is to explore how SSL can be performed for cases where corresponding labeled instances with noisy initial labels. For example, the labeling of samples may be contributed by the community, so we can only obtain noisy labels for the training dataset. One solution to this problem is [237], which augments the prediction objective with consistency where the same prediction is made given similar percepts. Based on graph SSL, [238] introduces a new L_1-norm formulation of Laplacian regularization inspired by sparse coding. [239] deals with this problem from the perspective of memorization effects, which proposed a learning paradigm combining co-training and mean teacher.

Robust semi-supervised learning. The common of the latest state-of-the-art approaches is the application of consistency training on augmented unlabeled data without changing the model predictions. One attempt is made by VAT [171] and VAdD [174]. Both of them employ adversarial training to find the optimal adversarial example. Another promising approach is data augmentation (adding noise or random perturbations, CutOut [230], RandomErasing [240], HideAndSeek [241] and GridMask [242]), especially advanced data augmentation, such as AutoAugment [176], RandAugment [219], CTAugment [63], and Mixup [228] which also can be considered as a form of regularization.

Safe semi-supervised learning. In SSL, it is generally accepted that unlabeled data can help improve learning performance, especially when labeled data is scarce. Although it is remarkable that unlabeled data can improve learning performance under appropriate assumptions or conditions, some empirical studies [243], [244], [245] have shown that the use of unlabeled data may lead to performance degeneration, making the generalization performance even worse than a model learned only with labeled data in real-world applications. Thus, safe semi-supervised learning approaches are desired, which never significantly degrades learning performance when unlabeled data is used.

9 Conclusion

Deep semi-supervised learning is a promising research field with important real-world applications. The success of deep learning approaches has led to the rapid growth of DSSL techniques. This survey provides a taxonomy of existing DSSL methods, and groups DSSL methods into five categories: Generative models, Consistency Regularization models, Graph-based models, Pseudo-labeling models, and Hybrid models. We provide illustrative figures to compare the differences between the approaches within the same category. Finally, we discuss the challenges of DSSL and some future research directions that are worth further studies.

References

[1] J. Padmanabhan and J. M. J. Premkumar, "Advanced deep neural networks for pattern recognition: An experimental study," in *SoCPaR*, ser. Advances in Intelligent Systems and Computing, vol. 614. Springer, 2016, pp. 166–175.

[2] K. Yun, A. Huyen, and T. Lu, "Deep neural networks for pattern recognition," *CoRR*, vol. abs/1809.09645, 2018.

[3] Q. Zhang, L. T. Yang, Z. Chen, and P. Li, "A survey on deep learning for big data," *Inf. Fusion*, vol. 42, pp. 146–157, 2018.

[4] T. Tran, T. Pham, G. Carneiro, L. J. Palmer, and I. D. Reid, "A bayesian data augmentation approach for learning deep models," in *NIPS*, 2017, pp. 2797–2806.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*. IEEE Computer Society, 2016, pp. 770–778.

[7] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[8] A. Torfi, R. A. Shirvani, Y. Keneshloo, N. Tavvaf, and E. A. Fox, "Natural language processing advancements by deep learning: A survey," *CoRR*, vol. abs/2003.01200, 2020.

[9] I. J. Goodfellow, Y. Bengio, and A. C. Courville, *Deep Learning*, ser. Adaptive computation and machine learning. MIT Press, 2016.

[10] Y. LeCun, Y. Bengio, and G. E. Hinton, "Deep learning," *Nat.*, vol. 521, no. 7553, pp. 436–444, 2015.

[11] O. Chapelle, B. Schölkopf, and A. Zien, "Introduction to semi-supervised learning," in *Semi-Supervised Learning*. The MIT Press, 2006, pp. 1–12.

[12] X. Zhu and A. B. Goldberg, *Introduction to Semi-Supervised Learning*, ser. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2009.

[13] A. Oliver, A. Odena, C. Raffel, E. D. Cubuk, and I. J. Goodfellow, "Realistic evaluation of deep semi-supervised learning algorithms," in *NeurIPS*, 2018, pp. 3239–3250.

[14] D. J. Miller and H. S. Uyar, "A mixture of experts classifier with learning based on both labelled and unlabelled data," in *NIPS*. MIT Press, 1996, pp. 571–577.

[15] K. Nigam, A. McCallum, S. Thrun, and T. M. Mitchell, "Text classification from labeled and unlabeled documents using EM," *Mach. Learn.*, vol. 39, no. 2/3, pp. 103–134, 2000.

[16] T. Joachims, "Transductive inference for text classification using support vector machines," in *ICML*. Morgan Kaufmann, 1999, pp. 200–209.

[17] K. P. Bennett and A. Demiriz, "Semi-supervised support vector machines," in *NIPS*. The MIT Press, 1998, pp. 368–374.

[18] X. Zhu, Z. Ghahramani, and J. D. Lafferty, "Semi-supervised learning using gaussian fields and harmonic functions," in *ICML*. AAAI Press, 2003, pp. 912–919.

[19] M. Belkin, P. Niyogi, and V. Sindhwani, "Manifold regularization: A geometric framework for learning from labeled and unlabeled examples," *J. Mach. Learn. Res.*, vol. 7, pp. 2399–2434, 2006.

[20] A. Blum and S. Chawla, "Learning from labeled and unlabeled data using graph mincuts," in *ICML*. Morgan Kaufmann, 2001, pp. 19–26.

[21] D. Zhou, O. Bousquet, T. N. Lal, J. Weston, and B. Schölkopf, "Learning with local and global consistency," in *NIPS*. MIT Press, 2003, pp. 321–328.

[22] A. Blum and T. M. Mitchell, "Combining labeled and unlabeled data with co-training," in *COLT*. ACM, 1998, pp. 92–100.

[23] O. Chapelle, B. Schölkopf, and A. Zien, Eds., *Semi-Supervised Learning*. The MIT Press, 2006.

[24] G. Qi and J. Luo, "Small data challenges in big data era: A survey of recent progress on unsupervised and semi-supervised methods," *CoRR*, vol. abs/1903.11260, 2019.

[25] Y. Ouali, C. Hudelot, and M. Tami, "An overview of deep semi-supervised learning," *CoRR*, vol. abs/2006.05278, 2020.

[26] O. Chapelle and A. Zien, "Semi-supervised classification by low density separation," in *AISTATS*. Society for Artificial Intelligence and Statistics, 2005.

[27] A. K. Agrawala, "Learning with a probabilistic teacher," *IEEE Trans. Inf. Theory*, vol. 16, no. 4, pp. 373–379, 1970.

[28] S. C. Fralick, "Learning to recognize patterns without a teacher," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 57–64, 1967.

[29] H. J. S. III, "Probability of error of some adaptive pattern-recognition machines," *IEEE Trans. Inf. Theory*, vol. 11, no. 3, pp. 363–371, 1965.

[30] D. Yarowsky, "Unsupervised word sense disambiguation rivaling supervised methods," in *ACL*. Morgan Kaufmann Publishers / ACL, 1995, pp. 189–196.

[31] V. R. de Sa, "Learning classification with unlabeled data," in *NIPS*. Morgan Kaufmann, 1993, pp. 112–119.

[32] J. Vittaut, M. Amini, and P. Gallinari, "Learning classification with both labeled and unlabeled data," in *ECML*, ser. Lecture Notes in Computer Science, vol. 2430. Springer, 2002, pp. 468–479.

[33] O. Chapelle, M. Chi, and A. Zien, "A continuation method for semi-supervised svms," in *ICML*, ser. ACM International Conference Proceeding Series, vol. 148. ACM, 2006, pp. 185–192.

[34] X. Zhu, "Learning from labeled and unlabeled data with label propagation," *Tech Report*, 2002.

[35] Y. Bengio, O. Delalleau, and N. L. Roux, "Label propagation and quadratic criterion," in *Semi-Supervised Learning*. The MIT Press, 2006, pp. 192–216.

[36] J. Weston, F. Ratle, H. Mobahi, and R. Collobert, "Deep learning via semi-supervised embedding," in *Neural Networks: Tricks of the Trade (2nd ed.)*, ser. Lecture Notes in Computer Science. Springer, 2012, vol. 7700, pp. 639–655.

[37] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010.

[38] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu, "A survey on deep transfer learning," in *ICANN*, ser. Lecture Notes in Computer Science, vol. 11141. Springer, 2018, pp. 270–279.

[39] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, 2021.

[40] Y. Li, L. Guo, and Z. Zhou, "Towards safe weakly supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 334–346, 2021.

[41] T. M. Hospedales, A. Antoniou, P. Micaelli, and A. J. Storkey, "Meta-learning in neural networks: A survey," *CoRR*, vol. abs/2004.05439, 2020.

[42] H. Peng, "A comprehensive overview and survey of recent advances in meta-learning," *CoRR*, vol. abs/2004.11149, 2020.

[43] J. Vanschoren, "Meta-learning: A survey," *CoRR*, vol. abs/1810.03548, 2018.

[44] W. Yin, "Meta-learning for few-shot natural language processing: A survey," *CoRR*, vol. abs/2007.09604, 2020.

[45] M. Huisman, J. N. van Rijn, and A. Plaat, "A survey of deep meta-learning," *CoRR*, vol. abs/2010.03522, 2020.

[46] L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *CoRR*, vol. abs/1902.06162, 2019.

[47] X. Liu, F. Zhang, Z. Hou, Z. Wang, L. Mian, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *CoRR*, vol. abs/2006.08218, 2020.

[48] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *CoRR*, vol. abs/2011.00362, 2020.

[49] J. Jeong, S. Lee, J. Kim, and N. Kwak, "Consistency-based semi-supervised learning for object detection," in *NeurIPS*, 2019, pp. 10 758–10 767.

[50] N. Souly, C. Spampinato, and M. Shah, "Semi supervised semantic segmentation using generative adversarial network," in *ICCV*. IEEE Computer Society, 2017, pp. 5689–5697.

[51] Y. LeCun, "The mnist database of handwritten digits," *http://yann. lecun. com/exdb/mnist/*, 1998.

[52] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011.

[53] A. Coates, A. Y. Ng, and H. Lee, "An analysis of single-layer networks in unsupervised feature learning," in *AISTATS*, ser. JMLR Proceedings, vol. 15. JMLR.org, 2011, pp. 215–223.

[54] A. Krizhevsky, "Learning multiple layers of features from tiny images," in *https://www.cs.toronto.edu/ kriz/cifar.html/*, 2009.

[55] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *NIPS*, 2012, pp. 1106–1114.

[56] A. Rasmus, M. Berglund, M. Honkala, H. Valpola, and T. Raiko, "Semi-supervised learning with ladder networks," in *NIPS*, 2015, pp. 3546–3554.

[57] M. Sajjadi, M. Javanmardi, and T. Tasdizen, "Regularization with stochastic transformations and perturbations for deep semi-supervised learning," in *NIPS*, 2016, pp. 1163–1171.

[58] S. Laine and T. Aila, "Temporal ensembling for semi-supervised learning," in *ICLR*. OpenReview.net, 2017.

[59] A. Tarvainen and H. Valpola, "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results," in *NIPS*, 2017, pp. 1195–1204.

[60] L. Zhang and G. Qi, "WCP: worst-case perturbations for semi-supervised deep learning," in *CVPR*. IEEE, 2020, pp. 3911–3920.

[61] V. Verma, A. Lamb, J. Kannala, Y. Bengio, and D. Lopez-Paz, "Interpolation consistency training for semi-supervised learning," in *IJCAI*. ijcai.org, 2019, pp. 3635–3641.

[62] D. Berthelot, N. Carlini, I. J. Goodfellow, N. Papernot, A. Oliver, and C. Raffel, "Mixmatch: A holistic approach to semi-supervised learning," in *NeurIPS*, 2019, pp. 5050–5060.

[63] D. Berthelot, N. Carlini, E. D. Cubuk, A. Kurakin, K. Sohn, H. Zhang, and C. Raffel, "Remixmatch: Semi-supervised learning with distribution matching and augmentation anchoring," in *ICLR*. OpenReview.net, 2020.

[64] J. Li, R. Socher, and S. C. H. Hoi, "Dividemix: Learning with noisy labels as semi-supervised learning," in *ICLR*. OpenReview.net, 2020.

[65] M. Everingham, L. V. Gool, C. K. I. Williams, J. M. Winn, and A. Zisserman, "The pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.

[66] T. Lin, M. Maire, S. J. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: common objects in context," in *ECCV*, ser. Lecture Notes in Computer Science, vol. 8693. Springer, 2014, pp. 740–755.

[67] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and F. Li, "Imagenet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[68] Y. Tang, J. Wang, B. Gao, E. Dellandréa, R. J. Gaizauskas, and L. Chen, "Large scale semi-supervised object detection using visual and semantic knowledge transfer," in *CVPR*. IEEE Computer Society, 2016, pp. 2119–2128.

[69] J. Gao, J. Wang, S. Dai, L. Li, and R. Nevatia, "NOTE-RCNN: noise tolerant ensemble RCNN for semi-supervised object detection," in *ICCV*. IEEE, 2019, pp. 9507–9516.

[70] K. Sohn, Z. Zhang, C. Li, H. Zhang, C. Lee, and T. Pfister, "A simple semi-supervised learning framework for object detection," *CoRR*, vol. abs/2005.04757, 2020.

[71] S. Song, S. P. Lichtenberg, and J. Xiao, "SUN RGB-D: A RGB-D scene understanding benchmark suite," in *CVPR*. IEEE Computer Society, 2015, pp. 567–576.

[72] A. Dai, A. X. Chang, M. Savva, M. Halber, T. A. Funkhouser, and M. Nießner, "Scannet: Richly-annotated 3d reconstructions of indoor scenes," in *CVPR*. IEEE Computer Society, 2017, pp. 2432–2443.

[73] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the KITTI vision benchmark suite," in *CVPR*. IEEE Computer Society, 2012, pp. 3354–3361.

[74] N. Zhao, T. Chua, and G. H. Lee, "SESS: self-ensembling semi-supervised 3d object detection," in *CVPR*. IEEE, 2020, pp. 11 076–11 084.

[75] Y. S. Tang and G. H. Lee, "Transferable semi-supervised 3d object detection from RGB-D data," in *ICCV*. IEEE, 2019, pp. 1931–1940.

[76] J. Li, C. Xia, and X. Chen, "A benchmark dataset and saliency-guided stacked autoencoders for video-based salient object detection," *IEEE Trans. Image Process.*, vol. 27, no. 1, pp. 349–364, 2018.

[77] F. Perazzi, J. Pont-Tuset, B. McWilliams, L. V. Gool, M. H. Gross, and A. Sorkine-Hornung, "A benchmark dataset and evaluation methodology for video object segmentation," in *CVPR*. IEEE Computer Society, 2016, pp. 724–732.

[78] T. Brox and J. Malik, "Object segmentation by long term analysis of point trajectories," in *ECCV*, ser. Lecture Notes in Computer Science, vol. 6315. Springer, 2010, pp. 282–295.

[79] P. Yan, G. Li, Y. Xie, Z. Li, C. Wang, T. Chen, and L. Lin, "Semi-supervised video salient object detection using pseudo-labels," in *ICCV*. IEEE, 2019, pp. 7283–7292.

[80] R. Mottaghi, X. Chen, X. Liu, N. Cho, S. Lee, S. Fidler, R. Urtasun, and A. L. Yuille, "The role of context for object detection and semantic segmentation in the wild," in *CVPR*. IEEE Computer Society, 2014, pp. 891–898.

[81] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *CVPR*. IEEE Computer Society, 2016, pp. 3213–3223.

[82] G. J. Brostow, J. Shotton, J. Fauqueur, and R. Cipolla, "Segmentation and recognition using structure from motion point clouds," in *ECCV*, ser. Lecture Notes in Computer Science, vol. 5302. Springer, 2008, pp. 44–57.

[83] C. Liu, J. Yuen, and A. Torralba, "SIFT flow: Dense correspondence across scenes and its applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 5, pp. 978–994, 2011.

[84] J. Xiao, J. Hays, K. A. Ehinger, A. Oliva, and A. Torralba, "SUN database: Large-scale scene recognition from abbey to zoo," in *CVPR*. IEEE Computer Society, 2010, pp. 3485–3492.

[85] S. Gould, R. Fulton, and D. Koller, "Decomposing a scene into geometric and semantically consistent regions," in *ICCV*. IEEE Computer Society, 2009, pp. 1–8.

[86] J. Dai, K. He, and J. Sun, "Boxsup: Exploiting bounding boxes to supervise convolutional networks for semantic segmentation," in *ICCV*. IEEE Computer Society, 2015, pp. 1635–1643.

[87] S. Hong, H. Noh, and B. Han, "Decoupled deep neural network for semi-supervised semantic segmentation," in *NIPS*, 2015, pp. 1495–1503.

[88] G. Papandreou, L. Chen, K. P. Murphy, and A. L. Yuille, "Weakly- and semi-supervised learning of a deep convolutional network for semantic image segmentation," in *ICCV*. IEEE Computer Society, 2015, pp. 1742–1750.

[89] Y. Wei, H. Xiao, H. Shi, Z. Jie, J. Feng, and T. S. Huang, "Revisiting dilated convolution: A simple approach for weakly- and semi-supervised semantic segmentation," in *CVPR*. IEEE Computer Society, 2018, pp. 7268–7277.

[90] J. Lee, E. Kim, S. Lee, J. Lee, and S. Yoon, "Ficklenet: Weakly and semi-supervised semantic image segmentation using stochastic inference," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 5267–5276.

[91] S. Mittal, M. Tatarchenko, and T. Brox, "Semi-supervised semantic segmentation with high- and low-level consistency," *CoRR*, vol. abs/1908.05724, 2019.

[92] T. Kalluri, G. Varma, M. Chandraker, and C. V. Jawahar, "Universal semi-supervised semantic segmentation," in *ICCV*. IEEE, 2019, pp. 5258–5269.

[93] M. S. Ibrahim, A. Vahdat, M. Ranjbar, and W. G. Macready, "Semi-supervised semantic image segmentation with self-correcting networks," in *CVPR*. IEEE, 2020, pp. 12 712–12 722.

[94] Y. Ouali, C. Hudelot, and M. Tami, "Semi-supervised semantic segmentation with cross-consistency training," in *CVPR*. IEEE, 2020, pp. 12 671–12 681.

[95] X. Zhang, J. J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *NIPS*, 2015, pp. 649–657.

[96] P. N. Mendes, M. Jakob, and C. Bizer, "Dbpedia for nlp: A multilingual cross-domain knowledge base," in *Proceedings of the Eight International Conference on Language Resources and Evaluation (LREC'12)*, Istanbul, Turkey, May 2012.

[97] M. Chang, L. Ratinov, D. Roth, and V. Srikumar, "Importance of semantic representation: Dataless classification," in *AAAI*. AAAI Press, 2008, pp. 830–835.

[98] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *ACL*. The Association for Computer Linguistics, 2011, pp. 142–150.

[99] X. H. Phan, M. L. Nguyen, and S. Horiguchi, "Learning to classify short and sparse text & web with hidden topics from large-scale data collections," in *WWW*. ACM, 2008, pp. 91–100.

[100] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *AAAI*. AAAI Press, 2019, pp. 7370–7377.

[101] D. Vitale, P. Ferragina, and U. Scaiella, "Classification of short texts by deploying topical annotations," in *ECIR*, ser. Lecture Notes in Computer Science, vol. 7224. Springer, 2012, pp. 376–387.

[102] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," in *ACL*. The Association for Computer Linguistics, 2005, pp. 115–124.

[103] R. Johnson and T. Zhang, "Semi-supervised convolutional neural networks for text categorization via region embedding," in *NIPS*, 2015, pp. 919–927.

[104] D. D. Lewis, Y. Yang, T. G. Rose, and F. Li, "RCV1: A new benchmark collection for text categorization research," *J. Mach. Learn. Res.*, vol. 5, pp. 361–397, 2004.

[105] W. Xu, H. Sun, C. Deng, and Y. Tan, "Variational autoencoder for semi-supervised text classification," in *AAAI*. AAAI Press, 2017, pp. 3358–3364.

[106] T. Miyato, A. M. Dai, and I. J. Goodfellow, "Adversarial training methods for semi-supervised text classification," in *ICLR*. OpenReview.net, 2017.

[107] D. S. Sachan, M. Zaheer, and R. Salakhutdinov, "Revisiting LSTM networks for semi-supervised text classification via mixed objective function," in *AAAI*. AAAI Press, 2019, pp. 6940–6948.

[108] L. Hu, T. Yang, C. Shi, H. Ji, and X. Li, "Heterogeneous graph attention networks for semi-supervised short text classification," in *EMNLP/IJCNLP*. Association for Computational Linguistics, 2019, pp. 4820–4829.

[109] H. Jo and C. Cinarel, "Delta-training: Simple semi-supervised text classification using pretrained word embeddings," in *EMNLP/IJCNLP*. Association for Computational Linguistics, 2019, pp. 3456–3461.

[110] J. Chen, Z. Yang, and D. Yang, "Mixtext: Linguistically-informed interpolation of hidden space for semi-supervised text classification," in *ACL*. Association for Computational Linguistics, 2020, pp. 2147–2157.

[111] P. Budzianowski, T. Wen, B. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gasic, "Multiwoz - A large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 5016–5026.

[112] X. Huang, J. Qi, Y. Sun, and R. Zhang, "Semi-supervised dialogue policy learning via stochastic reward estimation," in *ACL*. Association for Computational Linguistics, 2020, pp. 660–670.

[113] C. Danescu-Niculescu-Mizil and L. Lee, "Chameleons in imagined conversations: A new approach to understanding coordination of linguistic style in dialogs," in *CMCL@ACL*. Association for Computational Linguistics, 2011, pp. 76–87.

[114] R. Lowe, N. Pow, I. Serban, and J. Pineau, "The ubuntu dialogue corpus: A large dataset for research in unstructured multi-turn dialogue systems," in *SIGDIAL Conference*. The Association for Computer Linguistics, 2015, pp. 285–294.

[115] J. Chang, R. He, L. Wang, X. Zhao, T. Yang, and R. Wang, "A semi-supervised stable variational network for promoting replier-consistency in dialogue generation," in *EMNLP/IJCNLP*. Association for Computational Linguistics, 2019, pp. 1920–1930.

[116] K. Lang, "Newsweeder: Learning to filter netnews," in *ICML*. Morgan Kaufmann, 1995, pp. 331–339.

[117] E. F. T. K. Sang and F. D. Meulder, "Introduction to the conll-2003 shared task: Language-independent named entity recognition," in *CoNLL*. ACL, 2003, pp. 142–147.

[118] E. F. T. K. Sang and S. Buchholz, "Introduction to the conll-2000 shared task chunking," in *CoNLL/LLL*. ACL, 2000, pp. 127–132.

[119] K. Gimpel, N. Schneider, B. O'Connor, D. Das, D. Mills, J. Eisenstein, M. Heilman, D. Yogatama, J. Flanigan, and N. A. Smith, "Part-of-speech tagging for twitter: Annotation, features, and experiments," in *ACL*. The Association for Computer Linguistics, 2011, pp. 42–47.

[120] O. Owoputi, B. O'Connor, C. Dyer, K. Gimpel, N. Schneider, and N. A. Smith, "Improved part-of-speech tagging for online conversational text with word clusters," in *HLT-NAACL*. The Association for Computational Linguistics, 2013, pp. 380–390.

[121] R. T. McDonald, J. Nivre, Y. Quirmbach-Brundage, Y. Goldberg, D. Das, K. Ganchev, K. B. Hall, S. Petrov, H. Zhang, O. Täckström, C. Bedini, N. B. Castelló, and J. Lee, "Universal dependency annotation for multilingual parsing," in *ACL*. The Association for Computer Linguistics, 2013, pp. 92–97.

[122] J. Hockenmaier and M. Steedman, "Ccgbank: A corpus of CCG derivations and dependency structures extracted from the penn treebank," *Comput. Linguistics*, vol. 33, no. 3, pp. 355–396, 2007.

[123] A. M. Dai and Q. V. Le, "Semi-supervised sequence learning," in *NIPS*, 2015, pp. 3079–3087.

[124] M. E. Peters, W. Ammar, C. Bhagavatula, and R. Power, "Semi-supervised sequence tagging with bidirectional language models," in *ACL*. Association for Computational Linguistics, 2017, pp. 1756–1765.

[125] M. Rei, "Semi-supervised multitask learning for sequence labeling," in *ACL*. Association for Computational Linguistics, 2017, pp. 2121–2130.

[126] K. Clark, M. Luong, C. D. Manning, and Q. V. Le, "Semi-supervised sequence modeling with cross-view training," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 1914–1925.

[127] J. Hajic, M. Ciaramita, R. Johansson, D. Kawahara, M. A. Martí, L. Màrquez, A. Meyers, J. Nivre, S. Padó, J. Stepánek, P. Stranák, M. Surdeanu, N. Xue, and Y. Zhang, "The conll-2009 shared task: Syntactic and semantic dependencies in multiple languages," in *CoNLL Shared Task*. ACL, 2009, pp. 1–18.

[128] S. Pradhan, A. Moschitti, N. Xue, H. T. Ng, A. Björkelund, O. Uryupina, Y. Zhang, and Z. Zhong, "Towards robust linguistic analysis using ontonotes," in *CoNLL*. ACL, 2013, pp. 143–152.

[129] M. G. Carneiro, T. H. Cupertino, L. Zhao, and J. L. G. Rosa, "Semi-supervised semantic role labeling for brazilian portuguese," *J. Inf. Data Manag.*, vol. 8, no. 2, pp. 117–130, 2017.

[130] S. V. Mehta, J. Y. Lee, and J. G. Carbonell, "Towards semi-supervised learning for deep semantic role labeling," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 4958–4963.

[131] R. Cai and M. Lapata, "Semi-supervised semantic role labeling with cross-view training," in *EMNLP/IJCNLP*. Association for Computational Linguistics, 2019, pp. 1018–1027.

[132] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100, 000+ questions for machine comprehension of text," in *EMNLP*. The Association for Computational Linguistics, 2016, pp. 2383–2392.

[133] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension," in *ACL*. Association for Computational Linguistics, 2017, pp. 1601–1611.

[134] J. Oh, K. Torisawa, C. Hashimoto, R. Iida, M. Tanaka, and J. Kloetzer, "A semi-supervised learning approach to why-question answering," in *AAAI*. AAAI Press, 2016, pp. 3022–3029.

[135] B. Dhingra, D. Pruthi, and D. Rajagopal, "Simple and effective semi-supervised question answering," in *NAACL-HLT*. Association for Computational Linguistics, 2018, pp. 582–587.

[136] S. Zhang and M. Bansal, "Addressing semantic drift in question generation for semi-supervised question answering," in *EMNLP/IJCNLP*. Association for Computational Linguistics, 2019, pp. 2495–2509.

[137] L. Schoneveld, "Semi-supervised learning with generative adversarial networks," in *Doctoral dissertation, Ph.D. Dissertation*, 2017.

[138] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. C. Courville, and Y. Bengio, "Generative adversarial nets," in *NIPS*, 2014, pp. 2672–2680.

[139] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *ICLR*, 2016.

[140] J. T. Springenberg, "Unsupervised and semi-supervised learning with categorical generative adversarial networks," in *ICLR*, 2016.

[141] E. L. Denton, S. Gross, and R. Fergus, "Semi-supervised learning with context-conditional generative adversarial networks," *CoRR*, vol. abs/1611.06430, 2016.

[142] A. Odena, "Semi-supervised learning with generative adversarial networks," *CoRR*, vol. abs/1606.01583, 2016.

[143] T. Salimans, I. J. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training gans," in *NIPS*, 2016, pp. 2226–2234.

[144] Z. Dai, Z. Yang, F. Yang, W. W. Cohen, and R. Salakhutdinov, "Good semi-supervised learning that requires a bad GAN," in *NIPS*, 2017, pp. 6510–6520.

[145] G. Qi, L. Zhang, H. Hu, M. Edraki, J. Wang, and X. Hua, "Global versus localized generative adversarial nets," in *CVPR*. IEEE Computer Society, 2018, pp. 1517–1525.

[146] X. Wei, B. Gong, Z. Liu, W. Lu, and L. Wang, "Improving the improved training of wasserstein gans: A consistency term and its dual effect," in *ICLR*. OpenReview.net, 2018.

[147] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," *CoRR*, vol. abs/1701.07875, 2017.

[148] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of wasserstein gans," in *NIPS*, 2017, pp. 5767–5777.

[149] J. Donahue, P. Krähenbühl, and T. Darrell, "Adversarial feature learning," in *ICLR*. OpenReview.net, 2017.

[150] V. Dumoulin, I. Belghazi, B. Poole, A. Lamb, M. Arjovsky, O. Mastropietro, and A. C. Courville, "Adversarially learned inference," in *ICLR*. OpenReview.net, 2017.

[151] A. Kumar, P. Sattigeri, and T. Fletcher, "Semi-supervised learning with gans: Manifold invariance with improved inference," in *NIPS*, 2017, pp. 5534–5544.

[152] C. Li, T. Xu, J. Zhu, and B. Zhang, "Triple generative adversarial nets," in *NIPS*, 2017, pp. 4088–4098.

[153] Z. Gan, L. Chen, W. Wang, Y. Pu, Y. Zhang, H. Liu, C. Li, and L. Carin, "Triangle generative adversarial networks," in *NIPS*, 2017, pp. 5247–5256.

[154] S. Wu, G. Deng, J. Li, R. Li, Z. Yu, and H. Wong, "Enhancing triplegan for semi-supervised conditional instance synthesis and classification," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 10 091–10 100.

[155] J. Dong and T. Lin, "Margingan: Adversarial training in semi-supervised learning," in *NeurIPS*, 2019, pp. 10 440–10 449.

[156] Z. Deng, H. Zhang, X. Liang, L. Yang, S. Xu, J. Zhu, and E. P. Xing, "Structured generative adversarial networks," in *NIPS*, 2017, pp. 3899–3909.

[157] Y. Liu, G. Deng, X. Zeng, S. Wu, Z. Yu, and H.-S. Wong, "Regularizing discriminative capability of cgans for semi-supervised generative learning," in *CVPR*, 2020.

[158] S. Yun, D. Han, S. Chun, S. J. Oh, Y. Yoo, and J. Choe, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *ICCV*. IEEE, 2019, pp. 6022–6031.

[159] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *ICLR*, 2014.

[160] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," in *ICML*, ser. JMLR Workshop and Conference Proceedings, vol. 32. JMLR.org, 2014, pp. 1278–1286.

[161] D. P. Kingma, S. Mohamed, D. J. Rezende, and M. Welling, "Semi-supervised learning with deep generative models," in *NIPS*, 2014, pp. 3581–3589.

[162] L. Maaløe, C. K. Sønderby, S. K. Sønderby, and O. Winther, "Auxiliary deep generative models," in *ICML*, ser. JMLR Workshop and Conference Proceedings, vol. 48. JMLR.org, 2016, pp. 1445–1453.

[163] M. E. Abbasnejad, A. R. Dick, and A. van den Hengel, "Infinite variational autoencoder for semi-supervised learning," in *CVPR*. IEEE Computer Society, 2017, pp. 781–790.

[164] S. Narayanaswamy, B. Paige, J. van de Meent, A. Desmaison, N. D. Goodman, P. Kohli, F. D. Wood, and P. H. S. Torr, "Learning disentangled representations with semi-supervised deep generative models," in *NIPS*, 2017, pp. 5925–5935.

[165] J. Schulman, N. Heess, T. Weber, and P. Abbeel, "Gradient estimation using stochastic computation graphs," in *NIPS*, 2015, pp. 3528–3536.

[166] Y. Li, Q. Pan, S. Wang, H. Peng, T. Yang, and E. Cambria, "Disentangled variational auto-encoder for semi-supervised learning," *Inf. Sci.*, vol. 482, pp. 73–85, 2019.

[167] T. Joy, S. M. Schmon, P. H. S. Torr, N. Siddharth, and T. Rainforth, "Rethinking semi-supervised learning in vaes," *CoRR*, vol. abs/2006.10102, 2020.

[168] M. Belkin and P. Niyogi, "Laplacian eigenmaps and spectral techniques for embedding and clustering," in *NIPS*. MIT Press, 2001, pp. 585–591.

[169] Z. Ke, D. Wang, Q. Yan, J. S. J. Ren, and R. W. H. Lau, "Dual student: Breaking the limits of the teacher in semi-supervised learning," in *ICCV*. IEEE, 2019, pp. 6727–6735.

[170] M. Pezeshki, L. Fan, P. Brakel, A. C. Courville, and Y. Bengio, "Deconstructing the ladder network architecture," in *ICML*, ser. JMLR Workshop and Conference Proceedings, vol. 48. JMLR.org, 2016, pp. 2368–2376.

[171] T. Miyato, S. Maeda, M. Koyama, and S. Ishii, "Virtual adversarial training: A regularization method for supervised and semi-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 8, pp. 1979–1993, 2019.

[172] P. Izmailov, D. Podoprikhin, T. Garipov, D. P. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," in *UAI*. AUAI Press, 2018, pp. 876–885.

[173] B. Athiwaratkun, M. Finzi, P. Izmailov, and A. G. Wilson, "There are many consistent explanations of unlabeled data: Why you should average," in *ICLR*. OpenReview.net, 2019.

[174] S. Park, J. Park, S. Shin, and I. Moon, "Adversarial dropout for supervised and semi-supervised learning," in *AAAI*. AAAI Press, 2018, pp. 3917–3924.

[175] Q. Xie, Z. Dai, E. H. Hovy, T. Luong, and Q. Le, "Unsupervised data augmentation for consistency training," in *NeurIPS*, 2020.

[176] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation strategies from data," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 113–123.

[177] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "Randaugment: Practical data augmentation with no separate search," *CoRR*, vol. abs/1909.13719, 2019.

[178] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in *ACL*. The Association for Computer Linguistics, 2016.

[179] S. Edunov, M. Ott, M. Auli, and D. Grangier, "Understanding back-translation at scale," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 489–500.

[180] X. Zhu, Z. Ghahramani, and J. D. Lafferty, "Semi-supervised learning using gaussian fields and harmonic functions," in *ICML*, 2003, pp. 912–919.

[181] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: online learning of social representations," in *KDD*. ACM, 2014, pp. 701–710.

[182] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: large-scale information network embedding," in *WWW*. ACM, 2015, pp. 1067–1077.

[183] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *KDD*. ACM, 2016, pp. 855–864.

[184] D. Wang, P. Cui, and W. Zhu, "Structural deep network embedding," in *KDD*. ACM, 2016, pp. 1225–1234.

[185] M. Belkin and P. Niyogi, "Laplacian eigenmaps and spectral techniques for embedding and clustering," in *NIPS*. MIT Press, 2001, pp. 585–591.

[186] S. Cao, W. Lu, and Q. Xu, "Deep neural networks for learning graph representations," in *AAAI*. AAAI Press, 2016, pp. 1145–1152.

[187] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric transitivity preserving graph embedding," in *KDD*. ACM, 2016, pp. 1105–1114.

[188] T. N. Kipf and M. Welling, "Variational graph auto-encoders," *CoRR*, vol. abs/1611.07308, 2016.

[189] ——, "Semi-supervised classification with graph convolutional networks," in *ICLR*. OpenReview.net, 2017.

[190] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1263–1272.

[191] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[192] Q. Li, Z. Han, and X. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *AAAI*. AAAI Press, 2018, pp. 3538–3545.

[193] R. Liao, M. Brockschmidt, D. Tarlow, A. L. Gaunt, R. Urtasun, and R. S. Zemel, "Graph partition neural networks for semi-supervised classification," in *ICLR*. OpenReview.net, 2018.

[194] Y. Zhang, S. Pal, M. Coates, and D. Üstebay, "Bayesian graph convolutional neural networks for semi-supervised classification," in *AAAI*. AAAI Press, 2019, pp. 5829–5836.

[195] S. Vashishth, P. Yadav, M. Bhandari, and P. P. Talukdar, "Confidence-based graph convolutional networks for semi-supervised learning," in *AISTATS*, ser. Proceedings of Machine Learning Research, vol. 89. PMLR, 2019, pp. 1792–1801.

[196] C. Zhuang and Q. Ma, "Dual graph convolutional networks for graph-based semi-supervised classification," in *WWW*. ACM, 2018, pp. 499–508.

[197] F. Hu, Y. Zhu, S. Wu, L. Wang, and T. Tan, "Hierarchical graph convolutional networks for semi-supervised node classification," in *IJCAI*. ijcai.org, 2019, pp. 4532–4539.

[198] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *ICLR*, 2015.

[199] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NIPS*, 2017, pp. 1024–1034.

[200] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated graph sequence neural networks," in *ICLR*, 2016.

[201] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014.

[202] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, "Learning a SAT solver from single-bit supervision," in *ICLR*. OpenReview.net, 2019.

[203] Z. Zhou and M. Li, "Semi-supervised learning by disagreement," *Knowl. Inf. Syst.*, vol. 24, no. 3, pp. 415–439, 2010.

[204] S. Qiao, W. Shen, Z. Zhang, B. Wang, and A. L. Yuille, "Deep co-training for semi-supervised image recognition," in *ECCV*, ser. Lecture Notes in Computer Science, vol. 11219. Springer, 2018, pp. 142–159.

[205] D. Chen, W. Wang, W. Gao, and Z. Zhou, "Tri-net for semi-supervised deep learning," in *IJCAI*. ijcai.org, 2018, pp. 2014–2020.

[206] Y. Cheng, X. Zhao, R. Cai, Z. Li, K. Huang, and Y. Rui, "Semi-supervised multimodal deep learning for RGB-D object recognition," in *IJCAI*. IJCAI/AAAI Press, 2016, pp. 3345–3351.

[207] R. Xia, C. Wang, X. Dai, and T. Li, "Co-training for semi-supervised sentiment classification based on dual-view bags-of-words representation," in *ACL*. The Association for Computer Linguistics, 2015, pp. 1054–1063.

[208] Z. Zhou and M. Li, "Tri-training: Exploiting unlabeled data using three classifiers," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 11, pp. 1529–1541, 2005.

[209] L. Breiman, "Randomizing outputs to increase prediction accuracy," *Mach. Learn.*, vol. 40, no. 3, pp. 229–242, 2000.

[210] H. He and X. Sun, "A unified model for cross-domain and semi-supervised named entity recognition in chinese social media," in *AAAI*. AAAI Press, 2017, pp. 3216–3222.

[211] Z. Zhang, F. Xing, X. Shi, and L. Yang, "Semicontour: A semi-supervised learning approach for contour detection," in *CVPR*. IEEE Computer Society, 2016, pp. 251–259.

[212] A. G. O. II, C. L. Giles, and D. Reitter, "Learning a deep hybrid model for semi-supervised text classification," in *EMNLP*. The Association for Computational Linguistics, 2015, pp. 471–481.

[213] I. Misra, A. Shrivastava, and M. Hebert, "Watch and learn: Semi-supervised learning of object detectors from videos," in *CVPR*. IEEE Computer Society, 2015, pp. 3593–3602.

[214] Y. Grandvalet and Y. Bengio, "Semi-supervised learning by entropy minimization," in *NIPS*, 2004, pp. 529–536.

[215] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *Workshop on challenges in representation learning, ICML*, vol. 3, no. 2, 2013.

[216] Q. Xie, M. Luong, E. H. Hovy, and Q. V. Le, "Self-training with noisy student improves imagenet classification," in *CVPR*. IEEE, 2020, pp. 10 684–10 695.

[217] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *CoRR*, vol. abs/1503.02531, 2015.

[218] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 6105–6114.

[219] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "Randaugment: Practical automated data augmentation with a reduced search space," in *CVPR Workshops*. IEEE, 2020, pp. 3008–3017.

[220] L. Beyer, X. Zhai, A. Oliver, and A. Kolesnikov, "S4L: self-supervised semi-supervised learning," in *ICCV*. IEEE, 2019, pp. 1476–1485.

[221] A. Kolesnikov, X. Zhai, and L. Beyer, "Revisiting self-supervised visual representation learning," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 1920–1929.

[222] S. Gidaris, P. Singh, and N. Komodakis, "Unsupervised representation learning by predicting image rotations," in *ICLR*. OpenReview.net, 2018.

[223] A. Dosovitskiy, J. T. Springenberg, M. A. Riedmiller, and T. Brox, "Discriminative unsupervised feature learning with convolutional neural networks," in *NIPS*, 2014, pp. 766–774.

[224] A. Dosovitskiy, P. Fischer, J. T. Springenberg, M. A. Riedmiller, and T. Brox, "Discriminative unsupervised feature learning with exemplar convolutional neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 9, pp. 1734–1747, 2016.

[225] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *CVPR*. IEEE Computer Society, 2015, pp. 1–9.

[226] A. Hermans, L. Beyer, and B. Leibe, "In defense of the triplet loss for person re-identification," *CoRR*, vol. abs/1703.07737, 2017.

[227] H. Pham, Q. Xie, Z. Dai, and Q. V. Le, "Meta pseudo labels," *CoRR*, vol. abs/2003.10580, 2020.

[228] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *ICLR*. OpenReview.net, 2018.

[229] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. Raffel, E. D. Cubuk, A. Kurakin, and C. Li, "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," in *NeurIPS*, 2020.

[230] T. Devries and G. W. Taylor, "Improved regularization of convolutional neural networks with cutout," *CoRR*, vol. abs/1708.04552, 2017.

[231] Z. Ren, R. A. Yeh, and A. G. Schwing, "Not all unlabeled data are equal: Learning to weight data in semi-supervised learning," in *NeurIPS*, 2020.

[232] Z. Hu, X. Ma, Z. Liu, E. H. Hovy, and E. P. Xing, "Harnessing deep neural networks with logic rules," in *ACL*. The Association for Computer Linguistics, 2016.

[233] Y. Tang, J. Wang, X. Wang, B. Gao, E. Dellandréa, R. J. Gaizauskas, and L. Chen, "Visual and semantic knowledge transfer for large scale semi-supervised object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 3045–3058, 2018.

[234] M. Qi, Y. Wang, J. Qin, and A. Li, "KE-GAN: knowledge embedded generative adversarial networks for semi-supervised scene parsing," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 5237–5246.

[235] S. Yu, X. Yang, and W. Zhang, "PKGCN: prior knowledge enhanced graph convolutional network for graph-based semi-supervised learning," *Int. J. Machine Learning & Cybernetics*, vol. 10, no. 11, pp. 3115–3127, 2019.

[236] P. Swarup, D. Chakrabarty, A. Sapru, H. Tulsiani, H. Arsikere, and S. Garimella, "Knowledge distillation and data selection for semi-supervised learning in CTC acoustic models," *CoRR*, vol. abs/2008.03923, 2020.

[237] S. E. Reed, H. Lee, D. Anguelov, C. Szegedy, D. Erhan, and A. Rabinovich, "Training deep neural networks on noisy labels with bootstrapping," in *ICLR*, 2015.

[238] Z. Lu and L. Wang, "Noise-robust semi-supervised learning via fast sparse coding," *Pattern Recognit.*, vol. 48, no. 2, pp. 605–612, 2015.

[239] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. W. Tsang, and M. Sugiyama, "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *NeurIPS*, 2018, pp. 8536–8546.

[240] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *AAAI*. AAAI Press, 2020, pp. 13 001–13 008.

[241] K. K. Singh, H. Yu, A. Sarmasi, G. Pradeep, and Y. J. Lee, "Hide-and-seek: A data augmentation technique for weakly-supervised localization and beyond," *CoRR*, vol. abs/1811.02545, 2018.

[242] P. Chen, S. Liu, H. Zhao, and J. Jia, "Gridmask data augmentation," *CoRR*, vol. abs/2001.04086, 2020.

[243] A. Singh, R. D. Nowak, and X. Zhu, "Unlabeled data: Now it helps, now it doesn't," in *NIPS*. Curran Associates, Inc., 2008, pp. 1513–1520.

[244] T. Yang and C. E. Priebe, "The effect of model misspecification on semi-supervised classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 10, pp. 2093–2103, 2011.

[245] N. V. Chawla and G. I. Karakoulas, "Learning from labeled and unlabeled data: An empirical study across techniques and domains," *J. Artif. Intell. Res.*, vol. 23, pp. 331–366, 2005.